

صلى الله عليه وسلم



دانشگاه صنعتی اصفهان

دانشکده مهندسی برق و کامپیوتر

توسعه برنامه دنبال کننده نقطه تمرکز چشم در متلب

استاد راهنما

دکتر بهزاد نظری

دانشجو

مهیار عنصری

شهریور ۱۴۰۰

تشکر و قدردانی

از پروردگار متعال و منان سپاسگذارم که کشتی وجود من را در دریای پرتلاطم زندگی هدایت نمود و از موج‌ها و سختی‌های آن به سلامت عبور داد.

از استاد راهنمای اینجانب، دکتر بهزاد نظری به خاطر همه راهنمایی‌ها و حمایت‌هایشان کمال تشکر را دارم و برای من همکاری با ایشان باعث افتخار بود.

از همه اعضای خانواده‌ام ممنونم که در همه حال و همه شرایط مثل کوه پشتیبان من بودند تا بر چالش‌های زندگی چیره شوم و از آنها عبور کنم.

از همه استادان ارجمند، معلمان دلسوز و دوستان عزیزم بابت نقش‌شان در زندگی خویش متشکرم.

فهرست مطالب

عنوان	صفحه
چکیده	۱
۱- فصل اول: مقدمه	۲
۲- فصل دوم : مفاهیم و تعاریف پیش نیاز	۳
۱-۲ آشنایی با پردازش تصویر	۳
۲-۲ زبان‌های برنامه‌نویسی برای پردازش تصویر	۴
۱-۲-۲ پایتون	۴
۲-۲-۲ متلب	۵
۳-۲ الگوریتم‌های لبه‌یابی	۶
۱-۳-۲ الگوریتم سوبل	۸
۲-۳-۲ الگوریتم کنی	۱۲
۴-۲ ساختار چشم و بینایی انسان	۱۵
۱-۴-۲ عنبیه	۱۶
۲-۴-۲ مردمک	۱۶
۵-۲ روش‌های مختلف تشخیص عنبیه و مردمک در تصاویر	۱۷
۱-۵-۲ روش داگمن	۱۷
۲-۵-۲ روش وایلدز	۱۷
۳-۵-۲ روش لی‌ما و همکاران	۱۸
۴-۵-۲ روش تیسه	۱۸
۵-۵-۲ روش جوشی و آگراوال	۱۸
۶-۲ تبدیل هاف	۱۹

۲۰	۱-۶-۲ تبدیل هاف دایروی
۲۱	۷-۲ کاربردهای تشخیص عنبیه و مردمک
۲۱	۱-۷-۲ سیستم‌های زیست‌سنج‌ای
۲۲	۲-۷-۲ کاربرد در پزشکی
۲۳	۳-۷-۲ تشخیص نگاه برای استفاده به عنوان موشواره و کنترل کامپیوتر
۲۴	۴-۷-۲ استفاده در طراحی تبلیغات برخط
۲۴	۵-۷-۲ کاربرد در رانندگی
۲۵	۳- فصل سوم: طراحی و پیاده‌سازی الگوریتم ردیاب چشم و نتایج
۲۵	۱-۳ فاز اول : تشخیص مردمک و عنبیه
۳۱	۲-۳ فاز دوم : کالیبراسیون
۳۶	۱-۲-۳ نتیجه گیری فاز دوم و برنامه‌های آینده
۳۷	منابع

چکیده

امروزه مسئله تشخیص نگاه از جمله مباحث مهم و به روز در پردازش تصویر است که در بسیاری از مسائل مانند کنترل رایانه و خودروهای هوشمند به کار می رود. برای تشخیص نگاه باید تصویری از چشم کاربر به سیستم داده شود و سیستم باید تخمین خود را از نقطه نگاه کاربر اعلام کند. این گزارش ابتدا برخی مفاهیم استفاده شده در پروژه را تعریف و معرفی می کند و در ادامه به بخش پیاده سازی و نتیجه گیری می پردازد. این پروژه دارای دو فاز است که در فاز اول به کمک لبه یاب های سوبل و کنی دایره دربرگیرنده عنبیه و مردمک تشخیص داده می شود و در فاز دوم با استفاده از بخشی از کد فاز اول پروژه و گرفتن چند تصویر از کاربر، کالیبراسیون صفحه نمایش با چشم کاربر انجام می شود. بعد از انجام کالیبراسیون، برنامه قادر است تا با دریافت تصاویر دلخواه از کاربر تشخیص دهد وی به کدام نقطه از صفحه نمایش نگاه می کرده است.

واژگان کلیدی: تشخیص نگاه - ردیابی چشم - تشخیص عنبیه - پردازش تصویر

۱- فصل اول: مقدمه

پردازش تصویر از مهم‌ترین و پربحث‌ترین موضوعات مطرح در مهندسی برق است. یکی از مباحثی که در این حوزه مطرح است، موضوع پردازش تصاویر چشم و به طور خاص، تشخیص نقطه‌ای است که چشم روی آن متمرکز است. درک نقطه‌ای که چشم اشخاص به آن نگاه می‌کند، کاربردهای زیادی دارد. معمولاً در مسائل تشخیص نگاه، به کمک دوربینی از چشم کاربر تصویربرداری می‌شود و سعی بر آن است تا با جداسازی اجزای مختلف چشم از یکدیگر، تصویر دریافتی را پردازش کرد. اگر عنبیه و مردمک با اندکی ارفاق دایره‌هایی کامل در نظر گرفته شوند، هدف و چالش مسئله تشخیص نگاه، جداسازی مرکز این دایره‌هاست.

به طور کلی می‌توان گفت که مرکز مردمک یا عنبیه همان جایی است که کاربر مشغول نگاه کردن است و به اصطلاح به آن سوی چشم^۱ گفته می‌شود. اما در حل این مسئله، دشواری‌ها و چالش‌های متعددی وجود دارد. یکی از مهمترین این مسائل، موضوع پردازش بی‌درنگ^۲ است که در سامانه‌هایی که نیاز به واکنشی فوری نسبت به تغییرات چشم شخص دارند، چالشی کلیدی به‌شمار می‌رود. همچنین کیفیت پایین تصاویر دریافتی از بعضی دوربین‌ها می‌تواند چالش دیگری در راه رسیدن به سوی چشم کاربر باشد. امکان دارد به دلیل شرایط محیطی مانند نور یا فاصله شخص و یا مسائل اقتصادی در طراحی، سامانه پردازشی نتواند تصویر درستی از چشم شخص دریافت کند که در این صورت تشخیص اجزای مختلف چشم از یکدیگر دشوار می‌شود.

امروزه سامانه‌های دنبال‌کننده چشم کاربردهای زیادی در مسائل مختلف پیدا کرده‌اند که در بخشی از فصل آینده به برخی از این کاربردها به صورت خلاصه پرداخته می‌شود.

^۱ Gaze

^۲ Real-Time Processing

۲- فصل دوم : مفاهیم و تعاریف پیش نیاز

۲-۱ آشنایی با پردازش تصویر

تابعی عددی^۱ از دو متغیر مستقل را می توان یک تصویر نامید. در حقیقت مقدار این تابع دوبعدی برای دو متغیر مستقل (x,y) سطح خاکستری^۲ تصویر در آن مختصات می باشد. بعضی از سیگنال ها ماهیت پیوسته دارند که به آن ها آنالوگ^۳ گفته می شود و بعضی دیگر ماهیت گسسته دارند که دیجیتال^۴ نام دارند. در حالی که پردازش تصاویر آنالوگ موضوعی قدیمی است و بعضاً زمینه نورشناسی^۵ را که ریشه ای باستانی دارد هم در این زمینه می گنجانند، پردازش تصاویر دیجیتال از اواسط دهه ۶۰ میلادی و با توسعه رایانه های نسل سوم شروع شد [1]. اگر مقادیر مختصات و مقدار تابع برای آن ها کمیت هایی گسسته مقدار باشند، به آن تصویر یک تصویر دیجیتال گفته می شود. به هر عضو سازنده یک تصویر دیجیتال، یک پیکسل گفته می شود. هر پیکسل همان گونه که قابل حدس است، دارای یک زوج مرتب برای مختصات و یک عدد برای شدت تصویر در آن مختصات است [2].

همان طور که در [2] آمده است، مرز مشخصی برای پردازش تصویر و زمینه های نزدیک آن وجود ندارد و نمی توان تعریفی برای محدود کردن این حوزه ها مشخص کرد. برای مثال بینایی کامپیوتر یکی از این حوزه ها است که هدف آن استفاده از رایانه برای شبیه سازی بینایی انسان است که این موضوع یکی از زیرشاخه های هوش مصنوعی محسوب می شود. زمینه دیگری به نام درک تصویر وجود دارد که بین بینایی کامپیوتر و پردازش تصویر قرار می گیرد.

به طور کلی می توان پردازش تصویر را در سه سطح تقسیم بندی کرد: پردازش های سطح پایین، پردازش های سطح میانی و پردازش های سطح بالا. اعمال ابتدایی که روی تصویر انجام می شوند معمولاً در دسته پردازش های سطح پایین جای می گیرند. از جمله آن ها می توان به بهبود کنتراست^۶ تصویر، تیز کردن لبه ها، کاهش یا حذف نویز موجود در تصویر و بزرگنمایی تصویر اشاره کرد که همگی در دسته بهبود تصویر جای دارند. همچنین بازسازی یا بازیابی تصویر

^۱ Scalar

^۲ Greyscale

^۳ Analog

^۴ Digital

^۵ Optics

^۶ Contrast

در دسته پردازش‌های سطح پایین قرار می‌گیرند. بازیابی تصویر حول تخمین یا ترمیم تصاویری است که توسط یک عامل خارجی دچار افت کیفیت شده‌اند، عواملی مثل نویز یا تاری که باعث می‌شوند تصویر کیفیت اولیه را نداشته باشد. همچنین بازسازی تصویر یکی از زیرشاخه‌های مهم بازیابی تصویر است که در تصویربرداری پزشکی و رادارها به کار می‌رود و اساس آن به این صورت است که برخی از ویژگی‌های تصویر به کمک دریافت چندین نمونه از تصویر بازسازی می‌شود.

معمولاً تبدیل تصاویر به عنوان پردازش سطح میانی محسوب می‌شود که در این دسته می‌توان به تبدیل‌های فوریه^۱، موجک^۲ و کسینوسی^۳ اشاره کرد. همچنین در [2] قطعه‌بندی تصویر را به عنوان مثالی از پردازش سطح میانی معرفی کرده و خاطر نشان کرده است که در پردازش سطح میانی، ورودی‌ها مثل پردازش سطح پایین تصویر هستند اما خروجی‌ها به جای اینکه تصویر باشند، ویژگی‌های استخراج شده از آن تصویر هستند. در نهایت اعمالی مثل کلاس‌بندی^۴ به عنوان پردازش سطح بالا محسوب می‌شوند.

۲-۲ زبان‌های برنامه‌نویسی برای پردازش تصویر

برای انجام پردازش‌های مختلف روی تصاویر، از زبان‌های برنامه‌نویسی مختلفی استفاده می‌شود. در ادامه به صورت مختصر زبان‌های متلب^۵ و پایتون^۶ و انجام پردازش تصویر در آن‌ها معرفی می‌شوند.

۲-۲-۱ پایتون

این زبان برنامه‌نویسی که در اوایل دهه ۹۰ میلادی مطرح شد، از جمله زبان‌های پرکاربرد است که یک زبان سطح بالا و شی‌گرا محسوب می‌شود. هم‌اکنون شرکت‌های بزرگی نظیر گوگل، یاهو، اینستاگرام و ناسا از این زبان در پروژه‌های خود استفاده می‌کنند.

امروزه این زبان برنامه‌نویسی در زمینه‌های بسیاری از قبیل طراحی سایت و بازی‌سازی و همچنین کاربردهای تخصصی بسته به کتابخانه مورد استفاده کاربر، مورد استفاده قرار می‌گیرد. از جمله این زمینه‌ها می‌توان به یادگیری

^۱ Fourier

^۲ Wavelet

^۳ Cosine

^۴ Classification

^۵ MATLAB

^۶ Python

ماشین، برنامه‌نویسی تحت وب و پردازش تصویر اشاره کرد که در ادامه تعدادی از کتابخانه‌های مخصوص پردازش تصویر در پایتون بررسی می‌شود:

- کتابخانه *Sickit-image*: این کتابخانه به کمک آرایه‌های *Numpy*، تصاویر دریافتی را تبدیل می‌کند. به دلیل اینکه این کتابخانه در زبان C نوشته شده است سرعت قابل توجهی دارد و به همین دلیل در پردازش تصویر فراگیر شده است.

- کتابخانه *OpenCV*: این کتابخانه که نخستین بار در سال ۲۰۰۰ میلادی و به زبان ++C نوشته شد، به دلیل سرعت بالایش مورد توجه قرار گرفته است. *OpenCV* که جزء اصلی‌ترین کتابخانه‌های پردازش تصویر در پایتون به شمار می‌رود، بیشتر در کنار کتابخانه‌های *NumPy*، *SciPy*، *Matplotlib* به کار می‌رود و تقریباً محبوب‌ترین کتابخانه برای پردازش تصویر به ویژه مباحث مرتبط با تشخیص شیء و تشخیص صورت محسوب می‌شود. [3]

- کتابخانه *Mahotas*: این کتابخانه می‌تواند با دریافت تصاویر دو بعدی و سه بعدی، آن‌ها را پردازش کرده و اطلاعاتی را از تصاویر استخراج نماید. همچنین این کتابخانه قابلیت انجام وظایفی نظیر ریخت‌شناسی، کانولوشن و تارزدایی را از تصویر دارد.

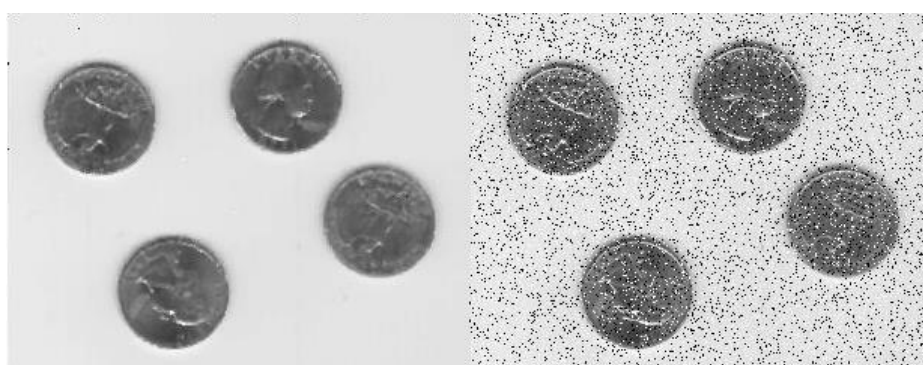
البته غیر از موارد فوق، کتابخانه‌های دیگری نیز برای پردازش تصویر در پایتون وجود دارد که مهم‌ترین و پرکاربردترین آن‌ها آورده شد.

۲-۲-۲ مطلب

این زبان برنامه‌نویسی که توسط شرکت *Mathworks* توسعه یافته است، به دلیل ساختار آرایه‌ای که دارد سرعت بسیار بالایی را در حل محاسبات به کاربر ارائه می‌کند و از این نظر، برای اجرای کارهای محاسباتی مناسب‌ترین زبان بین زبان‌های پرکاربرد برنامه‌نویسی است. این ساختار آرایه‌ای در کنار قابلیت رفع اشکال و ویرایش مناسب باعث شده تا مطلب به یک زبان برنامه‌نویسی عالی برای انجام تحقیقات و پژوهش و تدریس تبدیل شود.

همچنین دستورات آماده‌ای که در این زبان و یا جعبه‌ابزارهای آن وجود دارد باعث می‌شود تا کاربران راحت‌تر کار کنند و به جای صرف وقت برای نوشتن کد مدنظر خود، از این دستورات استفاده کرده و نتایج را به راحتی تفسیر نمایند. در ضمن وجود پارامترهای از پیش تعیین شده در هر دستور که در راهنمای مطلب موجود است، باعث می‌شود تا کاربر بتواند تحت شرایط مختلف داده‌های خود را مورد ارزیابی قرار دهد. مطلب به کاربران خود قابلیت تبدیل کد نوشته شده آنان به زبان‌های دیگر نظیر *C*، *Verilog* و ... را می‌دهد.

متلب در کنار پایتون یکی از مهم‌ترین زبان‌های برنامه‌نویسی برای انجام پردازش تصویر می‌باشد. از مسائل ساده‌ای مانند خواندن تصاویر با دستور *imread* و یا تغییر ماهیت رنگی تصویر از *rgb* به سطح خاکستری با دستور *rgb2gray* تا مباحث پیشرفته‌ای همچون پنهان نگاری یا تشخیص حالات صورت همگی به کمک متلب امکان‌پذیر است. از جعبه‌ابزار پردازش تصویر متلب در حل مسائلی نظیر نویز زدایی، تبدیل‌های مکانی روی تصویر، انجام تبدیل‌هایی مثل تبدیل فوریه سریع^۱، مدیریت رنگ و ... بهره گرفته می‌شود. همچنین به کمک این جعبه‌ابزار می‌توان بخش‌بندی تصاویر را انجام داد یا از هر تصویر ویژگی‌های دلخواه و موردنظر را استخراج کرد.



(ب)

(الف)

شکل ۲-۱ (الف): تصویر نویزدار ورودی متلب. (ب): تصویر نویززدایی شده خروجی متلب.

شکل ۲-۱ مثالی از یک عمل ساده روی تصویر می‌باشد. شکل ۲-۲ (الف)، تصویری است که با کمک دستور *imnoise* نویزی شده‌است و برای رفع نویز نمک و فلفل^۲، بهترین و راحت‌ترین راه استفاده از فیلتر میانه^۳ است. تصویر خروجی حاصل از اعمال این فیلتر بر روی تصویر نویزی در شکل ۲-۱ (ب) آمده‌است.

۲-۳ الگوریتم‌های لبه‌یابی

لبه‌یابی^۴ یکی از مسائل مهم در تشخیص و استخراج اشیا در تصاویر است [4]. عمده اهمیت لبه‌ها این است که مرز بین دو شیء یا دو محیط در یک تصویر، لبه‌های آن‌ها هستند و به کمک تشخیص لبه، می‌توان مسائلی نظیر تشخیص اشياء خاص یا بخش‌بندی تصاویر را انجام داد. در تعریف لبه ذکر این نکته ضروری است که لبه هیچ ضخامتی ندارد و مانند مرز یک سطح روشن و یک سطح تاریک است. به عبارت دیگر می‌توان لبه را به عنوان تغییر چشمگیر روشنایی

^۱ Fast Fourier Transform

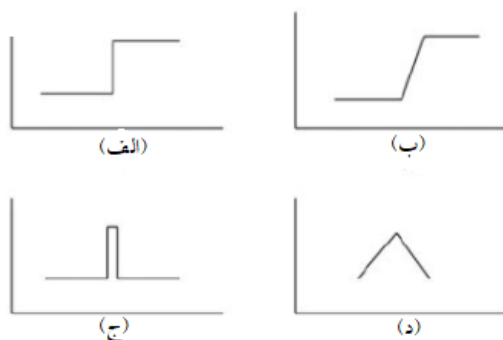
^۲ Salt and Pepper Noise

^۳ Median Filter

^۴ Edge Detection

در تصویر تعریف کرد [5]. برای لبه‌یابی روش‌ها و الگوریتم‌های مختلفی وجود دارد مانند الگوریتم‌های سوبل^۱، کنی^۲، پرویت^۳، رابرتز^۴، عبور از صفر^۵ و لاپلاسیان^۶.

بسیاری از مسائل مهم، مانند دید رباتیک، بخش‌بندی تصویر، استخراج ویژگی و فشرده‌سازی نیازمند تشخیص لبه به عنوان یکی از پایه‌ای‌ترین و ابتدایی‌ترین اعمال هستند. در مسائل لبه‌یابی، وجود نویز باعث می‌شود تا تشخیص لبه‌ها سخت‌تر باشد. در حقیقت می‌توان گفت در تصاویر بدون نویز که عمدتاً در کاربردهای عملی یافت نخواهند شد، اکثر الگوریتم‌ها می‌توانند به خوبی کار کنند اما در تصاویری که در دنیای واقعی گرفته می‌شوند و مشکلاتی از قبیل نویز و یا لبه‌های غیرایده‌آل دارند تشخیص لبه سخت‌تر است.



شکل ۲-۲: آشنایی با انواع لبه. (الف) پله، (ب) شیب، (ج) خط و (د) سقف.

شکل ۲-۲ انواع لبه را نمایش می‌دهد. لبه پله‌ای به صورت یک تغییر ناگهانی در مقادیر تابع شدت تصویر در طرفین یک ناپیوستگی تعریف می‌شود. اگر بعد از تغییر ناگهانی در تابع شدت تصویر، تابع دوباره به فاصله کمی به مقدار پیشین خود بازگردد به آن لبه، لبه خط گفته می‌شود. اما در واقعیت بنا به دلایل زیادی، این تغییرات فرکانس بالا کمتر اتفاق می‌افتد و در عمل این دو دسته از لبه دیده نمی‌شوند. در تصاویر واقعی تغییرات تیز کمتر دیده می‌شود و لبه پله به لبه شیب و لبه خط به لبه سقف تبدیل می‌شود و تغییرات به جای اینکه بلافاصله اتفاق بیفتند به صورت تدریجی تا فاصله‌ای محدود رخ می‌دهند.

^۱ Sobel

^۲ Canny

^۳ Prewitt

^۴ Roberts

^۵ Zero Crossing

^۶ Laplacian

به طور کلی می توان گفت الگوریتم لبه یاب خوب، الگوریتمی است که بتواند ضمن پیدا کردن لبه ها و مرزها به صورت دقیق، حذف نویز را نیز تا حد خوبی انجام دهد. در عمل نویززدایی از تصویر سخت است زیرا معمولاً برای حذف نویز از فیلترهای نرم کننده تصویر استفاده می شود که معمولاً یک تابع گاوسی است و اگر در تعریف این تابع گاوسی مقدار σ کم باشد باعث می شود تا دقت در لبه یابی بالا رود اما این مشکل را دارد که ممکن است نویزهایی را که روی تصویر افتاده است هم به عنوان لبه در نظر بگیرد و اگر مقدار σ زیاد باشد تصویر به شدت نرم و تار می شود و همین مسئله باعث می شود تا علیرغم حذف نویزها، به دلیل تار شدن تصویر، تشخیص لبه ها سخت شود. بعد از نرم کردن تصویر و حذف نویز با فیلتر گاوسی، یکی از مراحل کمک کننده در مسیر لبه یابی بهتر تعیین یک آستانه است تا لبه هایی که ضعیف هستند و ممکن است ناشی از باقی ماندن نویز باشند در نظر گرفته نشوند [6].

اگر به مسئله تشخیص لبه از دید ریاضی نگاه شود، توجه به این نکته ضروری است که اگر تصویر را به صورت تابع در نظر گرفته شود، چون لبه ها به عنوان تغییرات شدید تعریف شدند پس در صورت مشتق گرفتن از تابع شدت تصویر^۱، نقاط لبه به صورت قله یا دره در مشتق تابع شدت تصویر نمایان خواهند شد. همچنین الگوریتم لاپلاسن با استفاده از این نکته که اکستریم های مشتق اول، ریشه های مشتق دوم هستند ریشه های مشتق دوم را در تابع شدت تصویر پیدا می کند و آن ها را به عنوان لبه در نظر می گیرد. البته به دلیل اینکه الگوریتم لاپلاسن دوبار اقدام به مشتق گیری می کند بسیار به نویز حساس است و بهتر است ابتدا یک فیلتر گاوسی روی تصویر اعمال و بعد از این الگوریتم استفاده شود که به الگوریتم حاصل لاپلاسن گاوسی^۲ یا به اختصار *LOG* گفته می شود.

در ادامه دو الگوریتم پر کاربرد لبه یابی که در اجرای این پروژه نیز از آن ها استفاده شده است معرفی می شوند.

۱-۳-۲ الگوریتم سوبل

این الگوریتم که در سال ۱۹۶۸ میلادی توسط اروین سوبل^۳ و گری فلدمن^۴ توسعه یافت، بر محاسبه تخمین گرادیان تابع شدت تصویر بنا نهاده شده است. الگوریتم سوبل تصویر اصلی را با یک فیلتر عددی کوچک^۳ در ۳ در ۳ جهت افقی و عمودی کانوالو می کند و از نظر پیاده سازی و انجام محاسبات آسان و کم هزینه است. اما در عین حال این الگوریتم نقطه ضعف هایی دارد که از جمله آن می توان به ضعف در تغییرات فرکانس بالا اشاره کرد. به صورت کلی

^۱ Intensity Function

^۲ Laplacian of Gaussian

^۳ Erwin Sobel

^۴ Gary Feldman

این الگوریتم به دلیل اینکه از تخمین استفاده می کند لبه یاب قدرتمندی محسوب نمی شود اما به دلیل مزایایی که بالاتر برای آن ذکر شد از پرکاربردترین لبه یاب ها به شمار می رود.

1	2	1	-1	0	1
0	0	0	-2	0	2
-1	-2	-1	-1	0	1

(ب)

(الف)

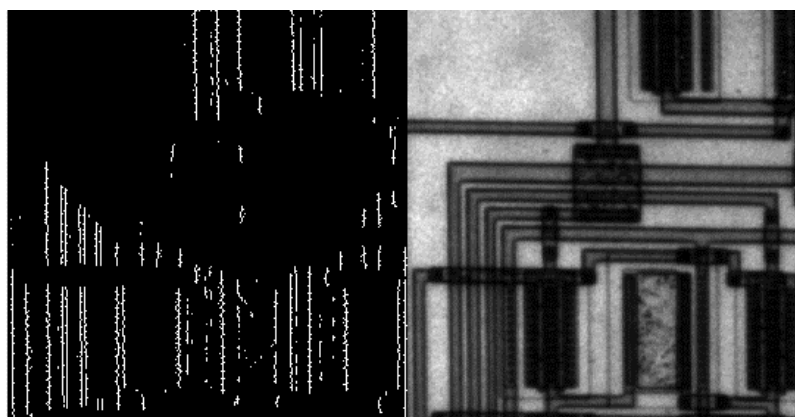
شکل ۲-۳: ماسک لبه یاب سوبل در جهت های (الف) افقی و (ب) عمودی .

برای لبه یابی در هر جهت، باید کانولوشن تصویر با ماسک مربوطه محاسبه شود. برای اینکه لبه ها در دو جهت محاسبه شوند باید یک بار در یکی از جهت ها لبه یابی انجام شود و سپس روی خروجی، در جهت دیگر لبه یابی شود. اگر حاصل کانولوشن تصویر با ماسک افقی و عمودی به ترتیب G_x و G_y نامیده شوند، اندازه و جهت لبه از روابط زیر به دست می آید:

$$G = \sqrt{G_x^2 + G_y^2} \approx |G_x| + |G_y|, \quad \theta = \tan^{-1} \frac{G_y}{G_x}$$

لازم به ذکر است که در تمامی کانولوشن ها میان یک ماسک (مانند ماسک سوبل) و تصویر، برای اینکه تصویر خروجی با تصویر اصلی هم اندازه باشد باید روی تصویر اصلی پوشش با صفر^۱ انجام شود. برای انجام کانولوشن با یک ماسک $n \times n$ (که n عددی فرد است) کافی است تا $\frac{n-1}{2}$ لایه صفر دور تا دور تصویر اصلی قرار داده شود. پوشش با صفر به منظور حفظ همه اطلاعات انجام می شود ولی اگر انجام نشود هم چون معمولاً در یک ردیف پیکسل کناری تصویر اطلاعات زیادی وجود ندارد، حجم کمی از اطلاعات از دست می رود. لبه یاب سوبل بیشتر در جاهایی به کار می رود که تغییرات شدت نور بین دو محیط یا دو شیء در تصویر زیاد باشد [7]. این لبه یاب به مانند اکثر لبه یاب هایی که از تکنیک نرم سازی و مشتق گیری استفاده می کند، گاهی اوقات بعضی از نویزها را به عنوان لبه حساب می کند و همچنین به دلیل ساختار ماسک های این الگوریتم، الگوریتم سوبل بر نقاط نزدیک مرکز ماسک تأکید و تمرکز بیشتری دارد (برخلاف ماسک پرویت که در تمامی نقاط یکسان است).

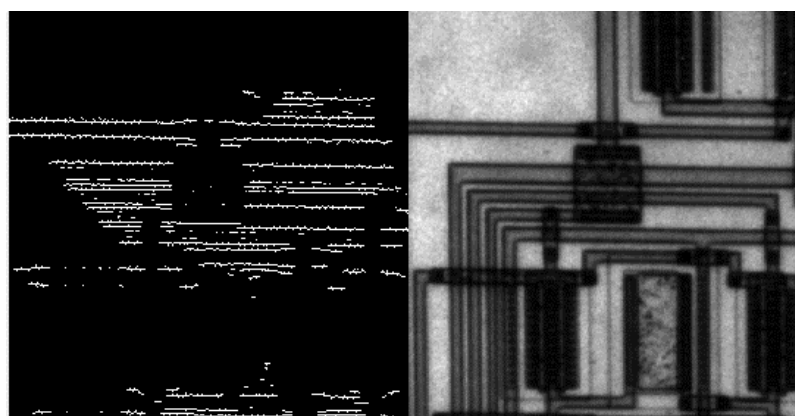
^۱ Zero Padding



(ب)

(الف)

شکل ۲-۴ (الف): تصویر ورودی. (ب): تصویر خروجی ناشی از اعمال لبه یاب سوبل در جهت عمودی.

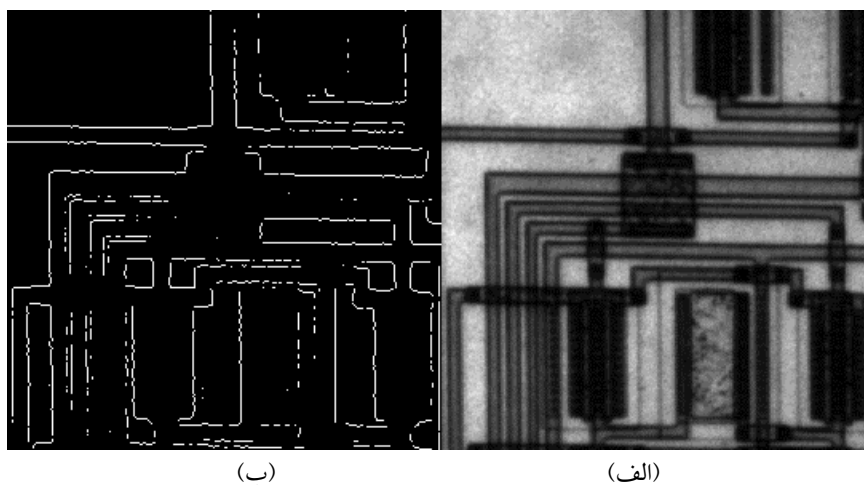


(ب)

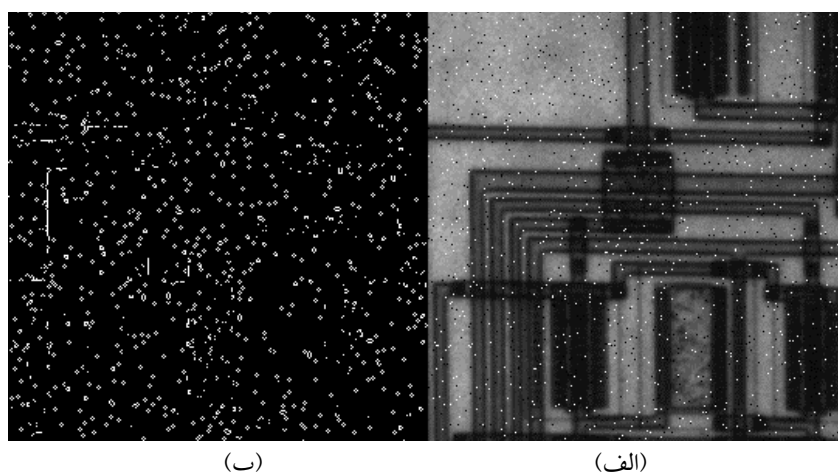
(الف)

شکل ۲-۵ (الف): تصویر ورودی. (ب): تصویر خروجی ناشی از اعمال لبه یاب سوبل در جهت افقی.

در شکل ۲-۴ و ۲-۵، تصاویر حاصله از اعمال ماسک لبه یاب عمودی و افقی سوبل در کنار تصویر اصلی آورده شده است. همان گونه که مشخص است لبه یاب در یک جهت در بسیاری از موارد کاربردی ندارد و باید هر دو ماسک روی تصویر اعمال شود تا بتوان به درستی با کمک لبه های تشخیص داده شده، مرزها را تشکیل داد و تصویر را قطعه بندی کرد یا مثلاً تبدیلات مختلف روی آن انجام داد. در صفحه بعد نتیجه اعمال ماسک در دو جهت روی تصویر و همچنین اعمال ماسک روی تصویر نوین دار آورده شده است.



شکل ۲-۶ (الف): تصویر ورودی. (ب): تصویر خروجی ناشی از اعمال لبه یاب سوبل در دو جهت افقی و عمودی.



شکل ۲-۷ (الف): تصویر نویزی ورودی. (ب): تصویر خروجی ناشی از اعمال لبه یاب سوبل در دو جهت افقی و عمودی.

همان گونه که بالاتر به آن پرداخته شد، الگوریتم سوبل در برابر تصویر نویزی عملکرد مناسبی ندارد و نمی تواند به درستی عمل لبه یابی را انجام دهد. اما با اعمال فیلتر گausسی روی تصویر نویزی و انجام مجدد لبه یابی مشاهده می شود که با تقریب خوبی لبه یابی درست انجام می شود.

۲-۳-۲ الگوریتم کنی

لبه یاب کنی که در اواخر دهه ۸۰ میلادی توسط جان اف. کنی^۱ پیشنهاد شد و توسعه یافت، تقریباً یکی از دقیق ترین و پرکاربردترین لبه یاب ها در بین عملگرهای لبه یاب موجود می باشد. این عملگر ساختار پیچیده تری نسبت به سایر عملگرها دارد و دارای یک الگوریتم چندمرحله ای برای تشخیص دقیق لبه است که با دقتی بالاتر از سایر عملگرها که به کمک محاسبه کانولوشن یک ماسک ماتریسی با تصویر، اقدام به لبه یابی می کردند لبه ها را تشخیص می دهد.

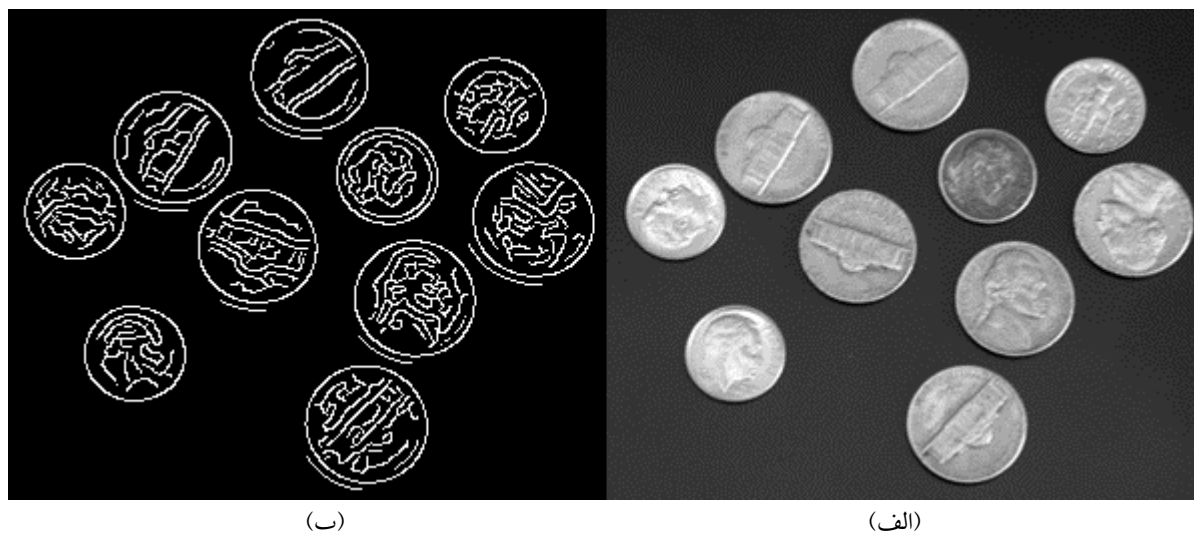
نخستین مرحله در اجرای الگوریتم کنی، اعمال یک فیلتر گاوسی دوبعدی به روی تصویر است. همان طور که پیشتر گفته شد اعمال فیلتر گاوسی باعث می شود تا در ازای اندکی تار و مات شدن تصویر، نویز موجود تا حد قابل قبولی حذف شود. ذکر این نکته لازم است که حتماً باید تصویر ورودی در حالت سطح خاکستری باشد. در گام بعدی به کمک گرادیان تصویر، نقاطی که دارای بیشترین تغییرات هستند و به عبارتی اکسترم های محلی گرادیان تابع شدت تصویر هستند به عنوان نقاطی که می توانند لبه در نظر گرفته شوند مشخص می شوند. اکنون باید نقاط مشخص شده را از نظر قدرت لبه بررسی و لبه های تشخیص داده شده ناشی از نویز را حذف کرد [7,8].

برای بررسی نقاط مشخص شده، دو آستانه که به صورت تجربی تعریف می شوند به عنوان آستانه های بالا و پایین در نظر گرفته می شوند و آن ها را با T_H و T_L نمایش می دهند. اگر در نقطه (x,y) مقدار گرادیان تابع شدت تصویر از T_H بیشتر بود آن گاه این نقطه به عنوان یک لبه قوی در نظر گرفته می شود. اگر هم نقطه ای وجود داشت که گرادیان در آن نقطه از مقدار T_L کمتر شد، قطعاً آن نقطه لبه نبوده است و از چرخه حذف می شود. اما اگر نقطه ای وجود داشته باشد که مقدار گرادیان تابع شدت تصویر در آن جا از T_H کمتر و از T_L بیشتر باشد، این نقطه یک لبه ضعیف نامیده می شود که می تواند یک لبه ضعیف موجود در تصویر اصلی باشد یا صرفاً نویزی باشد که عملگر را به خطا انداخته است. برای تشخیص دادن این دو دسته از نقاط از یکدیگر، از لبه های قوی استفاده می شود. به استفاده از لبه های قوی برای تشخیص لبه های ضعیف، لبه یابی به وسیله هیستریزیس^۲ گفته می شود. برای لبه یابی به وسیله هیستریزیس از این فرض بهره گرفته می شود که معمولاً لبه های ضعیف در امتداد و نزدیکی لبه های قوی تر قرار دارند. یک پیکسل که آن در مرحله قبل به عنوان لبه ضعیف شناخته شده بود انتخاب و ۸ پیکسل مجاورش بررسی می شود. اگر الگوریتم یکی از این ۸ پیکسل را قبلاً به عنوان پیکسلی از یک لبه قوی تشخیص داده بود، این لبه ضعیف به عنوان لبه در نظر گرفته شده و در غیر این صورت این لبه ناشی از نویز بوده و از نتایج حذف می شود [8]. لازم به ذکر است که در تعریف همسایگی

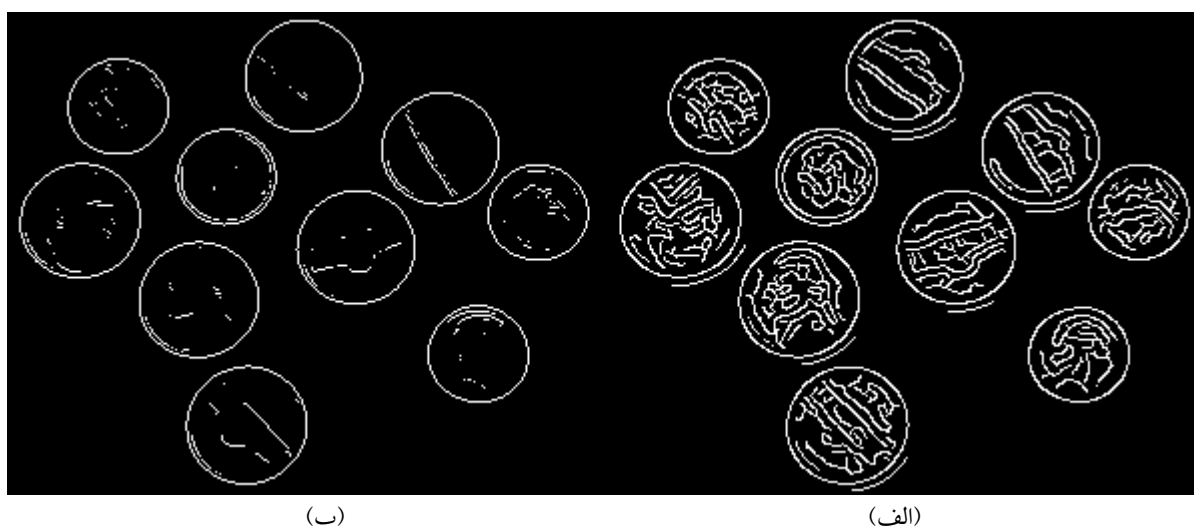
^۱ John F. Canny

^۲ Hysteresis

و اندازه و ابعاد آن اختلاف نظرهایی وجود دارد اما همسایگی ۸ پیکسلی، مقبول‌ترین و شایع‌ترین روش در نظر گرفتن همسایگی می‌باشد.



شکل ۲-۸ (الف): تصویر سکه‌ها به عنوان ورودی (ب): تصویر خروجی ناشی از اعمال لبه‌یاب کنی



شکل ۲-۹: تصویر خروجی ناشی از اعمال لبه‌یاب (الف) سوبل و (ب) کنی روی شکل ۲-۱۰ (الف).

همان‌طور که در شکل‌های ۲-۸ و ۲-۹ قابل مشاهده است، لبه‌یاب کنی به طور مشخصی از لبه‌یاب سوبل قدرتمندتر است و این برتری هم در تشخیص لبه‌های قوی تصویر (مانند مرز سکه‌ها) و هم در تشخیص لبه‌های ضعیف‌تر قابل مشاهده است. کاملاً مشخص است که عملگر کنی تمامی لبه‌های ضعیف را که باعث ایجاد جزئیات تصویر

(مانند نقش و نگار روی سکه‌ها) می‌شوند پیدا کرده‌است. عملگر سوبل پیاده‌سازی سریع‌تر و آسان‌تری دارد و می‌توان در کاربردهایی که نیاز به جزئیات نیست، از عملگر سوبل استفاده کرد و کیفیت قابل‌قبولی را به دست آورد.

درست است که علیرغم گذشت نزدیک به چهار دهه از ارائه عملگر کنی همچنان این عملگر به عنوان یکی از دقیق‌ترین عملگرها در زمینه لبه‌یابی به کار می‌رود، اما در طول این سال‌ها نقطه‌ضعف‌هایی در این الگوریتم پیدا شده و راه‌حل‌هایی برای بهبود عملکرد آن ارائه شده است که در ادامه معایب این الگوریتم و راه‌حل‌های پیشنهادی هر معضل بررسی می‌شود.

در [9] اولین معضلی که برای لبه‌یاب کنی ذکر شده‌است، همان موضوع تار شدن تصویر در اثر اعمال یک فیلتر گاوسی است. در واقع پیشنهاد می‌شود به جای اینکه هم نویز و هم لبه با اعمال فیلتر گاوسی یکسان نرم شوند، به کمک تکنیک‌هایی سعی شود تا نرمی بیشتری روی نویز و نرمی کمتری روی لبه اعمال شود. در حقیقت باید به جای فیلتر گاوسی، از یک فیلتر وفقی استفاده کرد. اساس کار این فیلتر وفقی این‌گونه است که برای هر پیکسل، ناپیوستگی موجود با پیکسل‌های مجاور محاسبه می‌شود و هرچه میزان این ناپیوستگی بیشتر باشد، احتمال اینکه آن پیکسل مربوط به یک لبه باشد بیشتر است و بنابراین باید فیلتر در آنجا مقدار کمتری داشته‌باشد. اما اگر میزان ناپیوستگی کم باشد یعنی احتمالاً آن نقطه لبه نبوده و ناشی از نویز یا تغییرات رنگ بوده‌است و بنابراین فیلتر در آن پیکسل مقدار زیادی خواهد داشت تا تصویر در آنجا بیشتر نرم شود.

یکی دیگر از مشکلاتی که در [9] و [10] به آن اشاره شده‌است، تعیین دستی آستانه‌های بالا و پایین است که ممکن است باعث کاهش دقت اجرای الگوریتم شود. برای رفع این مشکل باید از الگوریتم‌های تعیین آستانه خودکار مانند الگوریتم اتسو^۱ استفاده کرد. اساس کلی این الگوریتم‌ها مبتنی بر تقسیم تصویر به دو بخش پس‌زمینه و پیش‌زمینه با استفاده از یک آستانه اولیه است و سپس با انجام اعمالی مثل میانگین‌گیری از دو بخش پس‌زمینه و پیش‌زمینه این آستانه به‌روز می‌شود.

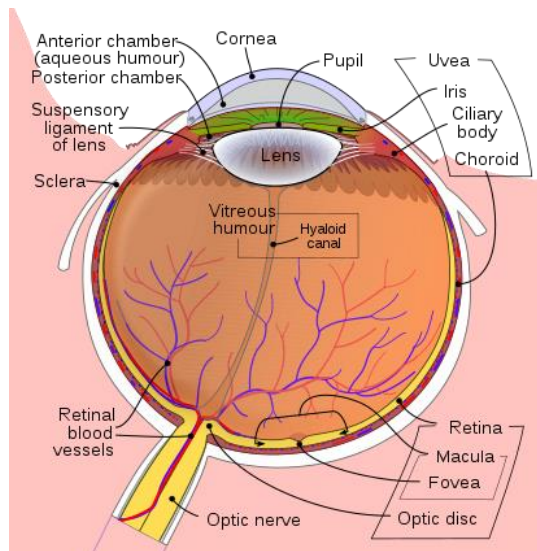
در حالی که خود الگوریتم کنی از عملگر ۳در۳ سوبل برای پیدا کردن اندازه و جهت گرادیان تابع شدت تصویر استفاده می‌کند، [10] استفاده از عملگر پرویت را توصیه می‌کند. به صورت کلی می‌توان گفت الزاماً استفاده از عملگر ۳در۳ سوبل منتج به بهترین نتیجه نخواهد شد و بهتر است تا متناسب با هدف لبه‌یابی، عملگر مرحله دوم کنی انتخاب شود و ممکن است عملگرهای رابرتز، پرویت و یا ۵در۵ سوبل در کاربردهای خاص بهتر عمل کنند.

^۱ Otsu

۲-۲ ساختار چشم و بینایی انسان

چشم انسان یک شکل تقریباً کروی است که از بخش‌های مختلفی تشکیل شده است. صلبیه^۱ که لایه‌ای سفیدرنگ و محکم است، حدود ۸۰ درصد از سطح بیرونی کره چشم را در بر می‌گیرد و از اجزای آن در برابر صدمات خارجی محافظت می‌کند. در جلوی کره چشم، این لایه محافظ شفاف است و قرنیه^۲ نام دارد. قرنیه که حدود ۱۱ میلی‌متر قطر دارد، همانند صلبیه وظیفه محافظت از اجزای چشم را دارد. اما این عضو یک وظیفه مهم‌تر دارد که وظیفه اصلی آن به‌شمار می‌رود و آن، اجازه دادن به نور برای ورود به لایه‌های درونی چشم و به عبارتی، هدایت نور به درون ساختمان چشم است. عنبیه^۳ که در پشت قرنیه قرار دارد و قسمت رنگی چشم است که از بیرون دیده می‌شود، دارای دو عضله است که با منبسط و منقبض شدنشان، مسیر ورودی نور از محیط بیرون به چشم از طریق مردمک^۴ را به طور طبیعی کنترل می‌کنند که شدت نور زیاد و مخرب وارد چشم نشود [11]. بخش بعدی به معرفی دقیق‌تر عنبیه و مردمک خواهد پرداخت.

بعد از ورود نور به ساختمان چشم، عدسی به کمک شکستن پرتوهای نور، آن‌ها را روی شبکه متمرکز می‌کند تا تصویر درستی تشکیل شود. در حقیقت تحدب این عدسی یا به عبارت بهتر فاصله کانونی این عدسی بسته به فاصله جسم مدنظر تا چشم تغییر می‌کند که به این عمل تطابق گفته می‌شود.



شکل ۲-۱۰: ساختار چشم انسان.

Sclera^۱

Cornea^۲

Iris^۳

Pupil^۴

۲-۴-۱ عنیبه

همان‌طور که بالاتر ذکر شد، عنیبه پشت قرنیه قرار دارد و در وسط آن سوراخی به نام مردمک وجود دارد. ماهیچه‌های موجود در عنیبه بسته به اینکه شدت نور محیط چقدر است، با منقبض و منبسط شدن مردمک را تنگ و گشاد می‌کنند تا میزان نور ورودی به چشم تنظیم شود و مناسب باشد تا عمل بینایی به درستی انجام شود و همچنین نور بیش از حد نیز وارد چشم نشود.

همان‌گونه که در شکل ۲-۱۰ مشخص است، مسطح بودن عنیبه باعث شده‌است تا بین خود و قرنیه که عضوی محدب است فضایی ایجاد شود که آن فضا با زلالیه^۱ پر شود که مایعی است که وظیفه تأمین مواد غذایی و اکسیژن را برای عدسی و قرنیه را دارد. بخش جلویی عنیبه که به اتاقک پیشین^۲ متصل است، کوژ و بخش عقبی عنیبه که به اتاقک پسین^۳ و عدسی در تماس است کاو است.

عنیبه تقریباً دقیق‌ترین و بهترین ویژگی بیومتریکی بدن می‌باشد. در دهه ۸۰ میلادی بود که دانشمندان متوجه شدند که هیچ دو عنیبه مشابهی تاکنون مشاهده نشده‌است. این یعنی نه تنها هیچ دو انسانی (حتی دوقلوهای یکسان) تاکنون الگوی یکسان عنیبه نداشته‌اند، بلکه در هر انسان هم الگوی دو عنیبه متفاوت است و بنابراین تشخیص الگوی عنیبه قوی‌ترین و دقیق‌ترین ابزار تشخیص هویت است و حجم بالایی از اطلاعات را در خود گنجانده‌است. تشخیص و احراز هویت تنها یکی از کاربردهای تشخیص عنیبه می‌باشد. بخش ۲-۷ به معرفی دقیق‌تر این موضوع و همچنین سایر کاربردهای تشخیص عنیبه می‌پردازد.

۲-۴-۲ مردمک

همان‌گونه که در بخش‌های پیشین گفته شد، مردمک سوراخی است که در وسط عنیبه جای دارد و عنیبه با انقباض و انبساط عضلات کنترل‌کننده آن، می‌تواند شدت نور ورودی به درون چشم را تنظیم نماید. علتی که مردمک مشکی‌رنگ دیده می‌شود این است که سلول‌های موجود در شبکیه تمامی نور آن را جذب می‌کند و به همین دلیل هیچ نوری از آن بازتاب نمی‌شود و مشکی‌رنگ دیده می‌شود. مشکی دیده شدن مردمک در برخی از مسائل کاربرد دارد زیرا با پس‌زمینه تصویر چشم (شامل عنیبه و صلبیه) تفاوت رنگی آشکاری دارد و با توجه به اینکه مرکز مردمک جایی است که کاربر به آن نگاه می‌کند، به راحتی می‌توان در برخی از کاربردها مخصوصاً در تصویربرداری در فواصل نزدیک از این ویژگی مردمک استفاده کرد و سوی نگاه کاربر را تشخیص داد.

^۱ Aqueous Humour

^۲ Anterior Chamber

^۳ Posterior Chamber

۲-۵ روش‌های مختلف تشخیص عنبیه و مردمک در تصاویر

برای اینکه بتوان برای کاربردهای مختلف از اطلاعات موجود در تصویر چشم استفاده کرد، باید دایره مردمک و عنبیه تشخیص داده شود. برای تشخیص دایره عنبیه و مردمک در تصاویر روش‌های متعددی وجود دارد که این بخش به معرفی آن‌ها اختصاص دارد. تشخیص عنبیه خود در بحث‌های متعددی از جمله احراز هویت کاربرد دارد و تشخیص مردمک هم می‌تواند در سیستم‌های تعقیب‌کننده چشم به کار گرفته شود. همچنین اگر با تقریب خوبی مردمک و عنبیه دایره‌هایی هم‌مرکز در نظر گرفته شوند، می‌توان از پیدا کردن مختصات مرکز یکی از این دو برای تشخیص سوی نگاه کاربر استفاده کرد.

۲-۵-۱ روش داگمن

مشهورترین روش در زمینه تشخیص عنبیه، روش جان داگمن^۱ است. داگمن با استفاده از عملگر انتگرال-دیفرانسیل و همچنین یک فیلتر گاوسی برای نرم کردن تصویر، یک لبه‌یاب دایره‌ای را تعریف می‌کند. در این الگوریتم پلک‌های بالایی و پایینی دو کمان در نظر گرفته می‌شوند. در حقیقت عملگر تعریف‌شده داگمن، برای تعیین مرزهای داخلی و خارجی عنبیه و همچنین پلک‌ها اقدام به محاسبه مشتق جزئی میانگین شدت نقاط دایره می‌کند و خروجی را با ماسک گاوسی کانوالو می‌کند. در نهایت به کمک بیشترین اختلاف یافت‌شده بین دایره درونی و بیرونی، مرکز و شعاع عنبیه را پیدا می‌کند. در حالی که برای یافتن مرزهای عنبیه انتگرال روی یک سطح دایروی گرفته می‌شد، برای تعیین مکان پلک‌ها اقدام به انتگرال‌گیری روی یک سطح بسته سهموی می‌شود. لازم به ذکر است چون در این عملگر از مشتق استفاده می‌شود، باز هم حساسیت به نویز وجود دارد و چون الگوریتم در تمامی تصویر می‌گردد و همه نقاط و شعاع‌ها را امتحان می‌کند، امکان دارد در اثر وجود نویز دچار خطا شود [7, 12, 13]

۲-۵-۲ روش وایلدز

یکی دیگر از روش‌های مطرح در زمینه تشخیص عنبیه، روش پیشنهادی ریچارد وایلدز^۲ می‌باشد [12]. وایلدز در مرحله اول روش خود از یک الگوریتم لبه‌یابی گرادیان‌محور و یک تابع نرم‌کننده گاوسی کمک می‌گیرد و بعد از اعمال عملگر مدنظر و همچنین کانوالو کردن تصویر با تابع گاوسی، تبدیل هاف را اعمال می‌کند. بعد از اعمال تبدیل هاف، در هر کانتور برای هر (x, y) از سیستم رأی‌گیری استفاده می‌شود و برای یک نقطه مشخص و شعاع‌های مختلف اگر پیکسلی روی دایره با مرکزیت آن نقطه و شعاع خاص قرار داشت، یک رأی برای آن دایره محسوب می‌شود.

^۱ John Daugman

^۲ Richard Wildes

در نهایت دایره‌ای که بیشترین رأی را به خود اختصاص داده باشد به عنوان کانتور مطلوب در نظر گرفته می‌شود. در این مدل هر دو پلک با قوس سهمی مدل می‌شوند و از اطلاعات لبه‌یابی افقی برای یافتن آن‌ها استفاده می‌شود [13].

۲-۵-۳ روش لی ما و همکاران

لی ما^۱ و همکارانش در آزمایشگاه ملی تشخیص الگو در آکادمی علوم چین، چندین مقاله در مورد تشخیص عنیه در تصاویر داده‌اند. آن‌ها از روش نزدیکترین خط ویژگی^۲ که خود تعمیمی از روش نزدیکترین همسایه است، استفاده می‌کنند و به کمک تفاوت زیاد تابع شدت تصویر مردمک نسبت به پس‌زمینه‌اش و استفاده از تبدیل هاف و عملگرهای لبه‌یابی اقدام به تعیین مرکز مردمک و سپس عنیه می‌کنند [7, 4].

۲-۵-۴ روش تیشه

کریستل-لویی تیشه^۳ در [14] اقدام به اصلاح و سریع‌تر کردن الگوریتم داگمن کرده‌است. در حقیقت روش تیشه از یک عملگر انتگرال-دیفرانسیل و تبدیل هاف کمک می‌گیرد تا بتواند تا مرکز مردمک را پیدا کند. نقطه‌ای که به عنوان مرکز است و آن را با (x_0, y_0) نمایش می‌دهد را به عنوان تابعی از اجزای گرادینان مرتبه اول تصویر بیان می‌کند. سپس یک بار دیگر عملگر انتگرال-دیفرانسیل به تصویر اعمال می‌شود تا با دقت بیشتری به دنبال مرزهای موجود میان عنیه و مردمک و مختصات مرکز عنیه بگردد. آن‌طور که ادعا شده این روش از روش داگمن دارای سرعت بیشتری است اما برخلاف آن روش، نویز ناشی از پلک‌ها و مژه‌ها را در نظر نمی‌گیرد و در برابرشان مقاوم نیست [6, 4].

۲-۵-۵ روش جوشی و آگراوال

این روش برخلاف سایر روش‌های ذکر شده تاکنون از روش‌های مشهور و الگو در دنیا نیست و در سال ۲۰۱۵ توسط کاویتا جوشی^۴ و سونیل آگراوال^۵ پیشنهاد شده‌است اما بر طبق [15] گفته شده‌است که برای تصاویر پایگاه داده CASIA که تقریباً اصلی‌ترین پایگاه داده تصاویر چشم است و در این پروژه نیز از آن کمک گرفته شده‌است دقت ۱۰۰ درصدی در تشخیص عنیه را ارائه کرده‌است. آن‌ها در این روش به کمک لبه‌یاب کنی مرزهای درونی و بیرونی عنیه را تشخیص دادند و سپس با تبدیل موجک هار^۶ و فیلتر گابور^۷ اقدام به استخراج ویژگی نمودند.

Li Ma^۱

Nearest Feature Line^۲

Christel-Loic Tisse^۳

Kavita Joshi^۴

Sunil Agrawal^۵

Haar Wavelet Transform^۶

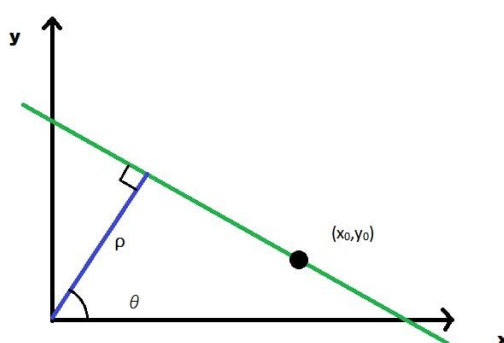
Gabor Filter^۷

روش های تشخیص مردمک هم غالباً بر تبدیل هاف و گرادیان گیری از تصاویر بنا نهاده شده اند و در اینجا به دلیل اینکه از عنیه در این پروژه استفاده بیشتری شد به آنان به طور مفصل پرداخته نشده است. در مجموع روش های مناسب متعددی برای تشخیص عنیه ارائه شده که در این پروژه از روش ارائه شده در [7] برای اجرای بخش اول پروژه که تشخیص عنیه و مردمک است استفاده شده است.

۲-۶ تبدیل هاف^۱

تبدیل هاف یک روش استخراج ویژگی پرکاربرد است که ایده اولیه اش در سال ۱۹۶۲ میلادی توسط پل هاف^۲ مطرح گشت. این روش که در ابتدا برای تشخیص خط در یک تصویر بود، بعدها برای تشخیص سایر اشکال مشخص مخصوصاً دایره و بیضی مورد استفاده قرار گرفت. در سال ۱۹۸۱ میلادی دانا بلرد^۳ تبدیل هاف تعمیم یافته را ارائه کرد. تفاوت تبدیل هاف کلاسیک و تبدیل هاف تعمیم یافته در آن بود که تبدیل هاف کلاسیک، تنها قادر به تشخیص اشکالی در تصویر بود که از قبل شکل تعریف شده و مشخصی مانند یک خط یا دایره داشتند و هدف استفاده از تبدیل هاف فقط موقعیت یابی شکل مذکور در تصویر بود. اما در روشی که بلرد ارائه کرد، هر شکل تصادفی ای صرفاً به کمک مدل توصیفی اش قابل تشخیص و پیدا کردن در تصویر بود.

در فضای دوبعدی، یک خط را می توان فقط با داشتن دو پارامتر شیب و عرض از مبدأ به صورت $y=mx$ توصیف کرد. در مختصات قطبی نیز خط را می توان با معادله $\rho = x \cos(\theta) + y \sin(\theta)$ نمایش داد. در این رابطه پارامتر ρ کمترین فاصله خط مذکور تا مبدأ و θ زاویه خط عمود بر خط مذکور با محور افقی است.



شکل ۲-۱۱: تصویری از بیان قطبی یک خط.

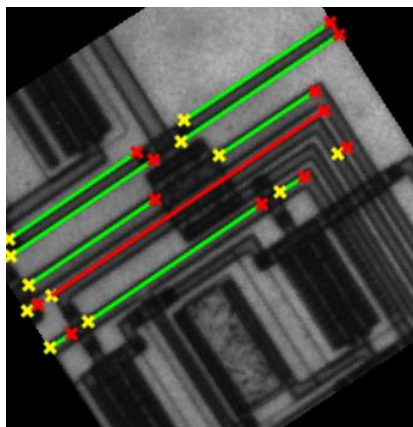
^۱ Hough Transform

^۲ Paul Hough

^۳ Dana Ballard

در [16] و [17] این گونه آمده است که تبدیل هاف از یک انباشتگر استفاده می کند تا تشخیص دهد برای یک ρ و θ مشخص، خطی در تصویر وجود دارد یا خیر. برای تبدیل هاف انباشتگر تعریف می شود. یعنی باید یک ماتریس دوبعدی با ابعاد ρ و θ تشکیل داد (می توان یک حدس اولیه با ابعاد بزرگ زد). برای هر پیکسل (x, y) موجود در تصویر، الگوریتم تبدیل هاف به دنبال این می گردد که آیا به صورت همزمان از آن پیکسل و یکی از همسایگانش خطی گذشته است یا خیر و در صورت مثبت بودن جواب، فرم قطبی خط را محاسبه می کند و بر درایه نظیر آن خط در انباشتگر یکی اضافه می کند. در نهایت وقتی همه پیکسل های تصویر را بررسی کرد، هر درایه از انباشتگر بیانگر تعداد پیکسل هایی است که بر روی خطی با آن مختصات و ویژگی قرار داشته است. بنابراین درایه هایی که مقدارشان زیاد است به احتمال زیاد نماینده یک خط با مختصات ρ و θ هستند. در نهایت باید یک سطح آستانه تعریف کرد تا درایه هایی که از آن مقدار بیشتر هستند به عنوان یک خط در صفحه در نظر گرفته شوند.

علتی که در تبدیل هاف از دستگاه مختصات قطبی به جای دکارتی برای درایه های انباشتگر استفاده می شود این است که در دستگاه دکارتی، x و y هر مقداری را از $-\infty$ تا ∞ می توانند داشته باشند اما ρ و θ بسته به دقتی که برایشان تعریف می شود محدود هستند و بی شمار مقدار نمی گیرند. [18]



شکل ۲-۱۲: خروجی تصویر پیش فرض *Circuit.tif* در متلب بعد از اجرای تبدیل هاف روی آن.

۲-۶-۱ تبدیل هاف دایروی

معادله دایره در دستگاه مختصات دکارتی به صورت $r^2 = (x-a)^2 + (y-b)^2$ است. در این معادله a و b مختصات مرکز دایره در جهت x و y هستند. می توان معادله فوق را به صورت دو معادله زیر بازنوشت:

$$x = a + R \cos(\theta), \quad y = b + R \sin(\theta)$$

به دلیل وجود سه پارامتر، انباشتگر تبدیل هاف دایروی نیز سه بعدی می شود. باز هم اینجا در ابتدا به کمک یکی از عملگرهای لبه یابی مانند کنی یا سوبل لبه ها تشخیص داده می شوند. سپس برای همه نقاطی که روی لبه ها هستند، با شعاع های مختلف r ، دایره رسم می شود. لازم به ذکر است که این دایره ها در دستگاه $a-b$ به جای $x-y$ کشیده می شوند. بعد از اینکه برای تمامی نقاط روی لبه با شعاع های مختلف دایره کشیده شد، آن گاه برای هر پیکسل در دستگاه جدید به ازای تعداد دوایر عبور کننده از آن به درایه نظیرش در انباشتگر افزوده می شود. بعد از تکمیل محاسبات، انباشتگر در هر درایه تعداد دایره های گذرنده از مختصات مذکور را دارد. در این جا برای هر شعاع، نقاطی که دارای بیشترین میزان در درایه خود هستند به عنوان یک مرکز برای دایره ای با شعاع مورد نظر انتخاب می شوند [18].

۷-۲ کاربردهای تشخیص عنیه و مردمک

این بخش به بررسی تعدادی از کاربردهای تشخیص عنیه و مردمک می پردازد. لازم به ذکر است که با تشخیص عنیه و مردمک، سوی نگاه کاربر نیز قابل تشخیص است و لذا کاربردهای تشخیص نگاه نیز در ادامه آورده شده است.

۱-۷-۲ سیستم های زیست سنجه ای

تشخیص و احراز هویت موضوعی است که همواره مورد توجه انسان بوده است. از قدیم بشر به دنبال این بوده تا بتواند در ارتباط های دو یا چند طرفه ای که دارد، هویت طرف مقابل را احراز کرده و از امنیت خود مطمئن شود. به طور مثال ارائه مدارک شناسایی مانند کارت ملی و یا امضای کاغذی از جمله مواردی هستند که با آن ها احراز هویت صورت می گیرد، اما این موارد به راحتی قابل جعل و شبیه سازی هستند و نمی توانند در مواردی که احراز هویت فرد اهمیت حیاتی دارد قابل اطمینان باشند. استفاده از مواردی مانند یک رمز مشخص هم نمی تواند قابل اتکا باشد چون در مواردی مانند شنود و یا حتی ارباب شخص قابل دست یابی است. پس باید به دنبال ویژگی هایی از شخص بود که در طول زمان و موقعیت های مختلف زندگی تغییر نکند. مواردی مثل اثر انگشت، الگوی عنیه، چهره شخص، شبکه، الگوی DNA و صدای فرد از این دست ویژگی ها هستند. به این ویژگی ها، ویژگی های زیست سنجه ای^۱ گفته می شود.

در بین این زیست سنجه ها، عنیه به دلایلی که ذکر خواهد شد از همه دقیق تر و قابل اطمینان تر است. مهم ترین نکته در مورد عنیه این است که الگوی عنیه شخص در طول زندگی ثابت می ماند و با افزایش سن شخص تغییری نمی کند. این در حالی است که مثلاً صدا یا چهره شخص ممکن است دچار تغییراتی شود که تشخیص را غیر ممکن یا سخت سازد. دومین نکته در مورد عنیه این است که نه تنها هیچ دو انسانی در جهان الگوی عنیه یکسانی ندارند، بلکه الگوی عنیه دو چشم یک انسان نیز با هم متفاوت است و همین مسئله باعث می شود تا در صورت تشخیص عنیه فرد هویت وی به سرعت احراز شود. یکی دیگر از ویژگی های مهم عنیه، محافظت شدن از آن در برابر آسیب ها به وسیله پلک

^۱ Biometrics

و قرنيه است. این مسئله به عنبيه در برابر اثر انگشت برتری می دهد زیرا در حالی که اثر انگشت هم مانند عنبيه احتمال تکرار بسیار پایینی دارد اما ممکن است در اثر حادثه ای دچار آسیب شود و مخدوش شود. یکی دیگر از ویژگی هایی که به عنبيه برتری می دهد، قابلیت تشخیص آن با فاصله است. برای تشخیص هویت شخص از طریق اثر انگشت نیاز است تا شخص انگشت خود را روی دستگاهی قرار دهد اما اگر دوربین با کیفیتی وجود داشته باشد می توان عنبيه را تا فاصله بین نیم تا یک متری تشخیص داد. همچنین باید اشاره کرد که عنبيه در اثر گریم یا آرایش چهره تغییر نمی کند و از این نظر به تشخیص چهره برتری دارد.

از نقاط ضعف عنبيه به عنوان یک زیست سنج می توان به ابعاد کوچک آن و همچنین امکان تغییر در صورت مبتلا شدن به برخی بیماری های خاص مانند التهاب عنبيه اشاره کرد. همچنین در صورت استفاده از لنزهای چشمی وضوح عنبيه کم می شود. قابل ذکر است این امکان وجود دارد تا با نصب دوربین های قوی و استفاده از پایگاه های داده، هویت افراد را در مکان های مختلف بدون رضایت شان تشخیص داد که این خطری بالقوه برای موضوع تشخیص هویت با عنبيه به شمار می رود زیرا مثلاً با هک شدن پایگاه داده و لو رفتن اطلاعات آن، هکرها می توانند با نصب دوربین ها در اماکن عمومی اطلاعات رفت و آمد مردم را استفاده کنند.

هر سامانه تشخیص هویتی نیاز به یک مرحله جمع آوری داده دارد. مثلاً در هندوستان، تصویر و اطلاعات عنبيه همه جمعیت ۱.۲ میلیارد نفری را به کمک روش داگمن که قبل تر به آن پرداخته شد جمع آوری کرده اند. اکنون از سامانه تشخیص هویتی مانند سامانه هندوستان انتظار می رود تا با دریافت تصویر عنبيه یک شخص هندی که از قبل در پایگاه داده این سامانه حضور داشته، بتواند هویت شخص را تشخیص و تأیید نماید.

۲-۷-۲ کاربرد در پزشکی

از مبحث تشخیص نگاه در پزشکی استفاده های زیادی می شود. مثلاً در [19] یک مدل سه بعدی را برای ردیابی چشم به منظور درمان غیرتهاجمی سرطان چشم پیشنهاد کرده اند. اکنون روش های درمانی سرطان لایه عروقی چشم^۱ که شامل عنبيه، جسم مژگانی و مشیمیه است به صورت تهاجمی هستند و برای موقعیت یابی تومور موجود در چشم نیازمند جراحی هستند. در این مقاله پیشنهاد شده است تا با استفاده از روش های تشخیص نگاه، پرتوهای پروتون را با چشم تنظیم کنند. بعد از اینکه شکل و موقعیت حدودی تومور تعیین شد، اکنون باید پرتوها را به گونه ای بتابانند تا از نقاط حساس چشم مانند لکه زرد، عصب بینایی و بخش جلویی چشم فاصله داشته باشد. برای تاباندن پرتوها بیمار را روی یک صندلی می نشانند و سرش را به کمک یک نگه دار چانه ثابت می کنند. سپس از بیمار خواسته می شود تا چشم خود را با یک نوار LED که در مقابل دیدگان وی قرار دارد تنظیم کند. به کمک ردیاب چشم، در هر لحظه به موقعیت

^۱ Uvea

اجزای چشم و تومور دسترسی وجود دارد. همچنین برای انجام این روش نیازمند یک کالیبراسیون نیز هستند که نگاهی بین دستگاه مختصات درون دستگاه و جهان بیرون انجام دهد. این عمل به کمک همان نور LED انجام می‌شود که از قبل کالیبره شده‌است. همچنین با انجام دادن کالیبراسیون به مختصات مرکز چشم‌ها دسترسی پیدا می‌شود که این مختصات در دستگاه برای درمان استفاده خواهد شد.

یکی دیگر از کاربردهایی که برای ردیابی چشم در پزشکی مطرح شده‌است، استفاده از این موضوع در تشخیص بیماری روان‌گسیختگی^۱ است. افرادی که دارای برخی از بیماری‌های روانپزشکی به ویژه روان‌گسیختگی هستند، قادر به انجام حرکات تعقیبی نرم چشم نیستند. این مسئله نه تنها در خود بیماران دیده می‌شود بلکه در بسیاری از موارد دیده‌شده که مشکل در انجام حرکات نرم چشم علاوه بر خود بیمار، خانواده درجه اول وی نیز دچار مشکل هستند و با توجه به این نکته می‌توان صحت این ادعا که بیماری‌های روان‌پزشکی مانند روان‌گسیختگی یا افسردگی ژنتیکی هستند را بررسی کرد. با بررسی داده‌های چشمی و رفتاری در کنار هم، ادعای وجود مشکل حرکت چشمی در بیماران دارای روان‌گسیختگی قوی‌تر می‌شود. [20]

۲-۷-۳ تشخیص نگاه برای استفاده به عنوان موشواره و کنترل کامپیوتر

امروزه و با گسترش تکنولوژی، سعی بر آن شده‌است تا دستگاه‌های تعاملی برای افراد دارای معلولیت ساخته‌شود. یکی از این دستگاه‌ها که برای این افراد می‌تواند مهم و حیاتی باشد، رایانه‌هایی هستند که از ردیابی چشم به جای موشواره و صفحه کلید استفاده می‌کنند. تحقیق روی این رایانه‌ها از اوایل دهه ۹۰ میلادی آغاز شد و در ابتدای قرن جدید میلادی به آرامی این رایانه‌ها از محیط آزمایشگاهی خارج و وارد بازار شدند اما همچنان برای بهبود این شرایط تلاش می‌شود تا بتوان رابط‌های کاربری بهتری را طراحی کرد. ردیاب Tobii یکی از معروف‌ترین سیستم‌های ردیابی چشم است که به کاربر اجازه می‌دهد بعد از کالیبراسیون و با کمک عینک مخصوص، در صفحات حرکت کرده یا آیکن‌ها را جابجا و انتخاب کند. در سال‌های اخیر سیستم‌هایی طراحی شده‌اند که به کاربر اجازه می‌دهند تا بتواند با استفاده از حرکات مشخص چشم مانند پلک‌زدن پیاپی و یا چشم‌ک‌زدن اعمالی مانند کلیک چپ یا جابجایی اجزای صفحه را انجام دهند.

یک روش جالب در [21] پیشنهاد شده‌است که بدون نیاز به سخت‌افزار خاصی و تنها با استفاده از وب‌کم و پایتون می‌توان به کاربر اجازه داد تا با حرکات خاصی بدون موشواره رایانه را کنترل کند. در الگوریتم پیشنهادی آن‌ها ابتدا تشخیص چهره و کالیبراسیون انجام می‌شود و سپس کاربر با باز کردن دهان الگوریتم را فعال می‌کند. برای حرکت

^۱ Schizophrenia

موشواره به چپ و راست کافی است تا کاربر سر را به طرف مورد نظر بچرخاند و برای *scroll* کردن باید کاربر به دورین خیره شود و سپس در منویی که باز می‌شود با حرکت سر گزینه مدنظر را انتخاب کند.

از چنین الگوریتم‌هایی می‌توان در طراحی سایر دستگاه‌های هوشمند نظیر تلفن‌های همراه یا تلویزیون یا ... نیز استفاده کرد تا بتوان شرایط زندگی بهتری را برای افراد دارای معلولیت فراهم کرد.

۲-۷-۴ استفاده در طراحی تبلیغات برخط

از روش‌های تشخیص نگاه می‌توان در طراحی مناسب تبلیغات اینترنتی استفاده کرد. با استفاده از ردیاب چشم می‌توان حدس زد که کاربر در حین استفاده از تلفن همراه و بازدید از صفحات اینترنتی بیشتر به چه نقاطی از صفحه نگاه می‌کند و با استفاده از همین مسئله می‌توان تبلیغات مناسب را طراحی کرد. همچنین می‌توان تخمین زد که کاربر بیشتر به چه رنگ‌هایی توجه می‌کند و تبلیغات را با توجه به آن طراحی نمود.

۲-۷-۵ کاربرد در رانندگی

به تازگی روش‌هایی طراحی شده‌است تا بتوان با استفاده از چند دورین حرکات چشم راننده را تحت نظر گرفت و از تصادفات رانندگی که اکثراً به دلیل عدم توجه راننده به جلو و پرت شدن حواس راننده صورت می‌گیرد جلوگیری کرد و آمار کشته‌ها را کاهش داد. در این جا هم نیاز به تشخیص چشم‌ها و کالیبراسیون نیاز است. مثلاً در خودروهای شرکت آئودی^۱ با استفاده از دورین مادون قرمز و دورین حرارتی و ... سعی بر آن شده‌است تا سیستم پیشنهاد استراحت به راننده طراحی شود و در برخی موارد از وقوع تصادفات نیز جلوگیری شود و همچنین برای باز کردن درب خودرو به سرنشین هشدار عبور خودروی دیگر را بدهد. در خودروهای شرکت کادیلاک^۲ سعی بر آن شده تا با استفاده از تشخیص و تعیین نگاه راننده میزان هوشیاری وی به کمک حرکت چشم‌ها و پلک‌ها سنجیده شود و واکنش مناسب نشان داده شود. توجه به این نکته ضروری است که در خودروهای هوشمند نیاز است تا چندین تکنیک و دورین برای تفسیر رفتار راننده و اتخاذ تصمیم مناسب به کار گرفته شود و صرفاً با ردیابی چشم نمی‌توان همه موارد را پیاده کرد [22].

^۱ Audi

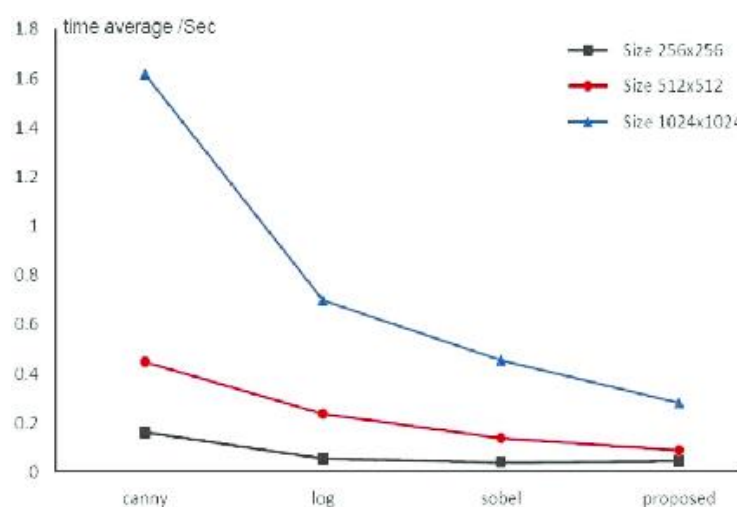
^۲ Cadillac

۳- فصل سوم: طراحی و پیاده‌سازی الگوریتم ردیاب چشم و نتایج

این فصل به توضیح دو بخش پروژه یعنی تشخیص مردمک و عنبیه و کالیبراسیون و نتایج به دست آمده خواهد پرداخت و نتایج به همراه روش‌های به کار گرفته شده در ادامه توضیح داده می‌شود.

۳-۱ فاز اول : تشخیص مردمک و عنبیه

در این فاز از پروژه، تصویر چشم از کاربر گرفته می‌شود. تصویر به عنوان ورودی به تابع *pupiliris* که برای انجام این پروژه نوشته شده است داده می‌شود. در این تابع برای تشخیص مردمک، ابتدا یک بار لبه‌یاب سوبل روی تصویر ورودی اعمال می‌شود. علت استفاده از لبه‌یاب سوبل این است که این لبه‌یاب علیرغم دقت نه‌چندان بالایی که در اثر تقریب دارد و در فصل قبل به آن اشاره شد، برای تشخیص اجسامی که تفاوت شدت نور زیادی با اجسام پس‌زمینه خود دارند به خوبی عمل می‌کند و چون مردمک با عنبیه و صلبیه تفاوت شدت نور بالایی دارد و به جزئیات در آن نیازی نیست از لبه‌یاب سوبل استفاده می‌شود. برای تصاویر ورودی پایگاه داده *CASIA* که ابعاد 320×280 را دارند و تصاویر سبکی محسوب می‌شوند استفاده از الگوریتم سوبل به جای الگوریتم کنی باعث می‌شود تا حدود نیم ثانیه زمان اجرا کمتر باشد.



شکل ۳-۱: نمودار مقایسه الگوریتم‌های مختلف لبه‌یابی از نظر زمان اجرا و ابعاد تصویر [24].

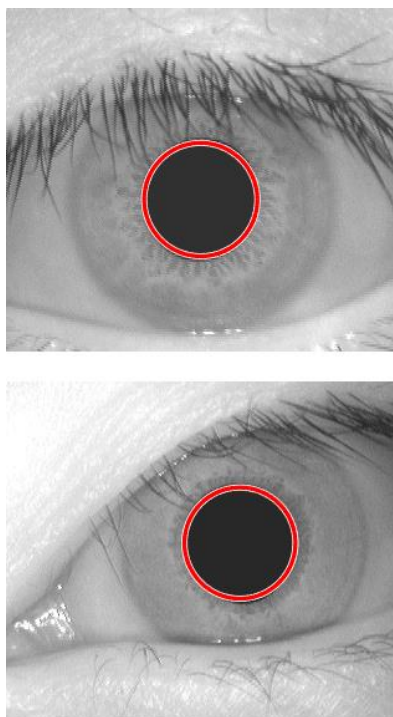
همان‌طور که در شکل ۲-۳ مشخص است، با افزایش اندازه ابعاد تصویر زمان اجرای الگوریتم کنی نسبت به سوبل به صورت نمایی زیاد می‌شود. به همین منظور در تشخیص مردمک از لبه‌یاب سوبل در دو جهت استفاده می‌شود.

بعد از اینکه لبه‌یاب سوبل به تصویر ورودی اعمال شد، از دستور *imfindcircles* استفاده می‌شود. این دستور با استفاده از تبدیل هاف اقدام به تشخیص دایره‌ها می‌کند. *imfindcircles* سه خروجی و چندین ورودی دارد. اولین خروجی آن، یک ماتریس با n سطر و ۲ ستون است که n تعداد دایره‌های تشخیص داده‌شده است و ستون‌ها مختصات مرکز هر دایره هستند. دومین خروجی یک بردار ستونی با n سطر است که درایه هر سطر، شعاع دایره سطر نظیر در ماتریس مراکز است. خروجی سوم، قدرت دایره تشخیص داده‌شده است که یک بردار ستونی با ابعاد مشابه بردار شعاع‌ها می‌باشد. هرچه این عدد بیشتر باشد دایره تشخیص داده‌شده قوی‌تر است. درایه‌های این بردار به صورت نزولی مرتب شده‌اند و درایه‌های دو ماتریس دیگر یعنی ماتریس مراکز و شعاع‌ها نیز بر اساس همین ترتیب چیده شده‌اند. یعنی سطر اول ماتریس مراکز، مختصات مرکز دایره‌ای است که بیشترین قدرت را دارد.

دستور *imfindcircles* حداقل نیاز به دو ورودی دارد. اولین ورودی تصویر مدنظر است و دومین ورودی آن، شعاع یا بازه شعاعی مدنظر است. اگر دومین ورودی شعاع دایره باشد، آنگاه این دستور به دنبال دایره‌ای می‌گردد که دقیقاً شعاع مدنظر را داشته‌باشد. در این مورد تنها مراکز دایره‌ها به عنوان خروجی مفید می‌تواند باشد زیرا همه دایره‌ها شعاع یکسان دارند و پارامتر قدرت هم کمکی نخواهد کرد. اما اگر مقدار دقیق شعاع نامشخص باشد و تخمینی از آن در دسترس باشد، دومین آرگومان دستور *imfindcircles* یک بازه شعاعی است که در تصویر به دنبال دایره‌هایی با شعاع عضو آن بازه شعاعی می‌گردد.

اگر از حالت پیش فرض این دستور استفاده شود، در خیلی از موارد ممکن است نتیجه دلخواه حاصل نشود. برای مثال شعاع مردمک یکی از تصاویر پایگاه داده CASIA را اندازه‌گیری کرده و همین تصویر به عنوان ورودی به دستور *imfindcircles* با همین آرگومان‌ها داده شد اما علیرغم حدس درست بازه شعاعی، دایره‌ای تشخیص داده‌نشد. این مسئله به این دلیل است که باید پارامترهای اسم-مقدار را در ورودی دستور *imfindcircles* تغییر داد. یکی از این پارامترها، *ObjectPolarity* نام دارد که مشخص می‌کند شکل دایره‌ای از پس‌زمینه‌اش روشن‌تر یا تاریک‌تر است. برای مردمک باید این مقدار را روی *Dark* قرار داده‌شود در حالی که پیش‌فرض روی *Bright* قرار دارد. یکی دیگر از پارامترهای ورودی که توجه به آن مهم است، فاکتور *Sensitivity* است. این پارامتر که عددی بین ۰ تا ۱ را اختیار می‌کند حساسیت دستور به اشکال دایره مانند را تعیین می‌کند. هرچه این مقدار به ۰ نزدیک‌تر باشد دستور به دنبال شکل‌هایی می‌گردد که بیشترین شباهت را با دایره ایده‌آل داشته باشد و به عبارتی دقت بالاتر می‌رود اما ممکن است باعث شود تا اشکالی که فرم دایره‌ای دارند اما کاملاً دایره نیستند (به مانند عنیبه و مردمک) تشخیص

داده نشوند. برای اکثر تصاویر پایگاه داده CASIA، مقدار ۰.۸۵ برای این فاکتور کافی به نظر می‌رسد زیرا مردمک تقریباً دایره کامل است و مرزهای واضحی هم دارد.



شکل ۳-۲: اجرای کد روی دو تصویر از تصاویر پایگاه داده CASIA برای پیدا کردن مردمک.

همان‌طور که در شکل ۳-۲ مشاهده می‌شود دایره مردمک با دقت خوبی پیدا شد. باید خاطر نشان کرد پیدا کردن مردمک، نیازمند داشتن یک تصویر با کیفیت از فاصله نزدیک از چشم است. همچنین باید در پارامتر بازه شعاعی تغییرات ایجاد کرد تا بتوان مردمک را پیدا کرد.

اکنون به بخش پیدا کردن عنبیه پرداخته می‌شود. برای پیدا کردن عنبیه باید به این نکته توجه کرد که برخلاف مردمک که با پس‌زمینه‌اش تفاوت آشکاری در شدت نور دارد، عنبیه در تصاویر سطح‌خاکستری تفاوت واضحی با صلبیه ندارد و مرز عنبیه با صلبیه لبه قوی‌ای ندارد و بنابراین باید تقویت شود. برای تقویت مرز عنبیه و صلبیه که از این به بعد مرز خارجی عنبیه نامیده می‌شود، باید از تفاضل دو تصویر استفاده کرد.

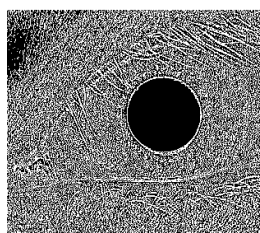
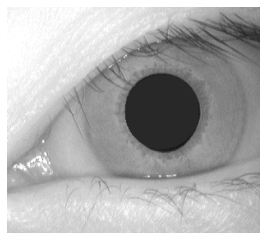
به این صورت که ابتدا در یک کپی از تصویر ابتدایی چشم، با استفاده از کانوالو کردن ماسک لاپلاسی با تصویر، تصویری حاصل می‌شود که درایه‌های آن در مرزهای قوی مقدار زیادی دارند و در جاهایی مانند مرز عنبیه و صلبیه که مرز قوی‌ای نیست مقدار درایه‌های تصویر کم است و می‌توان به همین دلیل گفت مرزهای قوی‌تر در تصویر خروجی کانولوشن تقویت می‌شود و سپس این تصویر تقویت‌شده از تصویر اولیه کم می‌شود. لبه‌ها در تصویر حاصل

از تفاضل تضعیف خواهند شد که این تضعیف در لبه‌های قوی بیشتر از لبه‌های ضعیف است و مرزهای قوی موجود در تصویر بیشتر از مرزهای ضعیفی مانند مرز خارجی عنیه تضعیف می‌شوند [7]. سپس با اعمال لبه‌یابی در تصویر آخر و انجام تبدیل هاف، می‌توان به حدود دایره عنیه دست پیدا کرد. در حقیقت روند پیدا کردن عنیه از روند پیدا کردن مردمک روند طولانی‌تر و پیچیده‌تری دارد. باید به این نکته توجه کرد که برای کسب نتیجه بهتر، باید نویز ناشی از مژه‌ها و پلک‌ها نیز حذف شود که در این روش از حذف آن‌ها اجتناب کرده و مشاهده می‌شود که همچنان با دقت خوبی دایره عنیه و مرکز آن به دست می‌آیند.

-1	-1	-1
-1	8	-1
-1	-1	-1

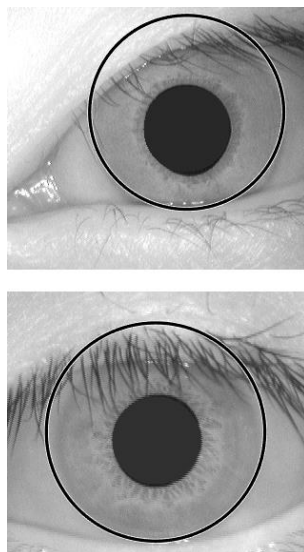
شکل ۳-۳: ماسک لاپلاسی.

عملگر لبه‌یابی لاپلاسین برخلاف خیلی از عملگرهای موجود مانند سوبل که دو ماسک در جهت‌های مختلف دارند، تنها یک ماسک دارد. برای تقویت مرزهای قوی از این عملگر استفاده می‌شود. دلیل استفاده از این عملگر این است که به دلیل ساختار این ماسک، پیکسلی که در مرکز قرار می‌گیرد به شدت تقویت می‌شود و پیکسل‌های اطرافش تضعیف می‌شوند. با اعمال این ماسک به تصویر، نقاطی که مقادیر بالایی دارند خود تقویت و همسایه‌های ضعیف‌تر آن‌ها تضعیف می‌شوند چون این مقدار بالا با ضریب منفی در ماسک حضور دارند و منجر به تضعیف می‌شود.



شکل ۳-۴: در بالا تصویر چشم ورودی و در پایین تصویر خروجی ناشی از اعمال ماسک لاپلاسین دیده می‌شوند.

اکنون تصویر حاصله را از تصویر اولیه کم می‌شود و روی تصویر حاصله، لبه‌یابی صورت می‌گیرد. برای این مرحله چون نیاز به دقت بالایی می‌باشد از الگوریتم لبه‌یابی کنی استفاده می‌شود. بعد از اعمال لبه‌یابی کنی به تصویر، به کمک دستور *imfindcircles* که مبتنی بر تبدیل هاف است، دایره‌ها پیدا می‌شوند. در تصاویر پایگاه داده CASIA برای اطمینان خاطر حاصل کردن از پیدا کردن دایره حدودی عنیبه، از نتایج خروجی با هر دو ورودی *Bright* و *Dark* که در صفحه ۲۶ ذکر شد استفاده می‌شود و بین قوی‌ترین دایره پیدا شده توسط هر کدام، مقایسه‌ای صورت می‌گیرد و دایره قوی‌تر به عنوان عنیبه اعلام می‌شود. با انجام این مقایسه تقریباً خطا کم می‌شود و به صفر نزدیک می‌شود. به صورت کلی استفاده از روش *Dark* برای تشخیص دایره عنیبه توصیه می‌شود چون در هر صورت عنیبه اندکی از صلیبه تیره‌تر است. در بررسی تصاویر پایگاه داده CASIA در مواردی خطا دیده شد که با استفاده همزمان از مقایسه نتایج برای دو حالت *Bright* و *Dark* به صفر رسید. اما در تصاویری که با فاصله از چشم گرفته می‌شود و مانند پایگاه داده CASIA تصویر خیلی نزدیک از چشم در دسترس نیست بهتر است تنها از روش *Dark* استفاده شود زیرا به تجربه خطاهایی در روش *Bright* مشاهده شد.



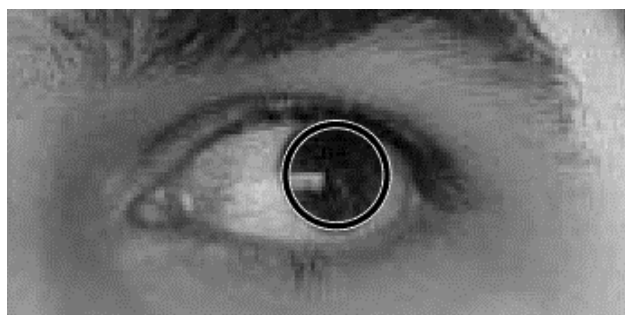
شکل ۳-۵: اجرای کد روی دو تصویر از تصاویر پایگاه داده CASIA برای پیدا کردن عنیبه .

با موفقیت عنیبه برای تصاویر پایگاه داده CASIA پیدا شد. در اینجا لازم به ذکر است که مانند مردمک، باید تقریب‌های مختلف از بازه شعاعی را امتحان کرد تا به تصویر دلخواه رسید. پارامتر *Sensitivity* برای تشخیص عنیبه مقداری از مردمک بیشتر است چون دایره ضعیف‌تری است و برای تصاویر CASIA حدود ۰.۹۷ یا ۰.۹۸ مناسب است.

برای بررسی صحت کد، روی دو تصویر از خارج از تصاویر پایگاه CASIA استفاده شد. نتایج به شرح زیر است:



شکل ۳-۶: اجرای کد روی تصویری از چشم در تصاویر غیر CASIA.



شکل ۳-۷: اجرای کد روی تصویری از چشم در تصاویر غیر CASIA.

مشاهده شد که کد پیدا کننده عنبیه در تصاویری غیر از تصاویر پایگاه داده CASIA مانند تصاویر موجود در شکل های ۳-۶ و ۳-۷ نیز درست کار می کند. به عنوان نکات و نتایج نهایی این بخش باید اعلام کرد که پارامترهای *Sensitivity* و *Radius Range* باید برای تصاویر مختلف تغییر پیدا کنند. مثلاً برای تصاویر پایگاه داده CASIA برای پیدا کردن مردمک $Sensitivity = 0.85$ و $Radius Range = [30 \ 55]$ و برای عنبیه

$Sensitivity = 0.98$ و $Radius Range = [90\ 160]$ نتایج خوبی را ارائه کرد. باید توجه کرد که طبق توضیحات نرم افزار متلب مقدار حداقل پارامتر $Radius Range$ در دستور *imfindcircles* نمی تواند از ۵ کمتر باشد و همچنین بهتر است تا اول و آخر این بازه نیز خیلی به هم نزدیک نباشند تا تشخیص دایره ها با دقت بالاتری صورت بگیرد. اما مثلاً برای تصاویر چشم که از فاصله حدود ۰.۵ متری و با دوربین خویش انداز گرفته شد، پارامترهای $Sensitivity = 0.96$ و $Radius Range = [10\ 60]$ تغییر کرد.

۲-۳ فاز دوم : کالیبراسیون

در این بخش از پروژه، سعی بر آن شده تا با دریافت ۴ تصویر از کاربر که به یکی از تصاویر تعریف شده برای کالیبراسیون نگاه می کند و محاسبه مختصات مرکز عنبیه شخص در تصویر که همان نقطه ای است که کاربر در حال نگاه کردن است، نگاشتی بین دو مجموعه مختصات نقاط صفحه نمایش و مجموعه مختصات مراکز عنبیه زده شود و در حقیقت صفحه نمایش با چشم کاربر کالیبره شود. پس از اتمام کالیبراسیون می توان از چشم تصویربرداری کرد و پس از تخمین مرکز عنبیه در تصویر تشخیص داد که کاربر به کدام نقطه صفحه نمایش نگاه می کرده است.

چون کاربر در مقابل لپ تاپ نشسته است و دوربین از او تصویربرداری می کند برای این که کاربر به صورت دستی مجبور به برش تصویر نشود، باید به کمک تابعی یک مستطیل دور یکی از چشم ها در نظر گرفته شده و بریده و جدا شود. این برش اتوماتیک دقت را بالا می برد زیرا برای حدس درست نقطه نگاه کاربر نیاز است تا کاملاً دستگاه مختصات در همه تصاویر ورودی کاربر یکسان باشد.

به همین منظور از تابع *sep* که به منظور انجام این فاز نوشته شده است، استفاده می شود. این تابع تصویر شخص در مقابل دوربین را از کاربر می گیرد و با شناسایی و موقعیت یابی چشم ها و محاسبه مختصات مرکز آن ها، بردار رسم مستطیل در دستور *imcrop* را برای رسم مستطیل دور یکی از چشم ها که در اینجا به صورت پیش فرض چشم چپ کاربر است (اما کاربر می تواند آن را به چشم راست تبدیل کند) به عنوان خروجی بر می گرداند. لازم به ذکر است دستور *imcrop* برای رسم یک مستطیل نیاز به دو آرگومان دارد که اولی تصویر ورودی است و دومی یک بردار است که به ترتیب شامل مؤلفه x و مؤلفه y برای رأس بالا-چپ مستطیل و طول و عرض آن مستطیل است. بعد از اینکه تابع *sep* مرکز دو چشم را (منظور مرکز شکل دو کی شکل چشم است) پیدا کرد، فاصله آن ها از هم حساب می شود. به تجربه دیده شد که یک مستطیل مناسب برای جدا کردن تصویر یک چشم، مستطیلی با طول فاصله بین مرکز دو چشم و عرض نصف آن می باشد. خروجی های تابع *sep* چهار عدد موجود در بردار ورودی *imcrop* هستند که فاصله بین مرکز دو چشم و نصف آن فاصله به عنوان سومین و چهارمین خروجی در نظر گرفته می شوند تا در دستور *imcrop* به عنوان طول و عرض کادر مستطیلی باشند. اما برای یافتن اولین و دومین خروجی تابع *sep* کافی است که نصف

طول و عرض محاسبه شده را از مرکز یکی از چشم‌ها کم کرده تا رأس بالا-چپ کادر دور چشم به دست آید و بردار رسم مستطیل دستور *imcrop* کامل شود.

نکته‌ای که باید به آن توجه شود این است که باید کادر در نظر گرفته شده برای تمامی چشم‌ها موقعیت یکسانی روی صورت داشته باشد تا مختصات همه مراکز روی یک دستگاه قرار بگیرد و عمل نگاشت قابل انجام باشد. همچنین به دلیل اینکه کاربر از دوربین فاصله دارد، پیدا کردن عنبیه راحت تر و دقیق تر از مردمک است و به همین منظور در این فاز دیگر به تشخیص مردمک پرداخته نمی شود و صرفاً مکان یابی عنبیه صورت می گیرد. با توجه به نکته ذکر شده، دو تابع *irisfinder1* و *irisfinder2* تعریف می شود.

در تابع *irisfinder1*، با استفاده از خروجی های تابع *sep* بردار رسم مستطیل دستور *imcrop* برگردانده می شود و سپس با فراخوانی تابع *imcrop* مستطیلی دور چشم چپ کشیده می شود و بعد با استفاده از کد فاز اول پروژه عنبیه پیدا می شود. نکته مهم اینجا است که خروجی های تابع *sep* به عنوان چهار مورد از خروجی تابع *irisfinder1* هم تعریف می شوند و به همراه مختصات مرکز عنبیه و شعاع آن شش خروجی این تابع را تشکیل می دهند که دوتای اول مختصات مرکز عنبیه و شعاع آن و چهار خروجی بعدی، آرگومان های لازم برای دستور *imcrop* هستند.

ساختار تابع *irisfinder2* شباهت زیادی به *irisfinder1* دارد و تنها تفاوت در این است که دیگر از تابع *sep* استفاده نمی شود بلکه آرگومان های دستور *imcrop* را به عنوان ورودی دریافت می کند و دیگر نیازی به صدا زدن تابع *sep* وجود ندارد. آرگومان های ورودی *irisfinder2* دقیقاً همان آرگومان های خروجی *irisfinder1* تعریف می شود تا همه کادرها روی یک بخش مشخص از صورت قرار بگیرند و همان طور که در بالا گفته شد همه مختصات مراکز عنبیه ها در یک صفحه و دستگاه باشد.

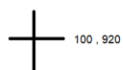
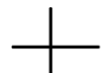
حال تابع *predictor* معرفی می شود که اصلی ترین تابع فاز دوم است. در این تابع ابتدا همه تصاویرهای لازم برای کالیبراسیون از کاربر گرفته می شود و سپس برای پیدا کردن مختصات عنبیه ها، برای یکی از تصاویر تابع *irisfinder1* صدا زده می شود و به کمک خروجی های آن، برای بقیه تصاویر تابع *irisfinder2* فراخوانی می شود. حال مختصات مرکز عنبیه در هر چهار تصویر کالیبراسیون به دست آمده است.

اکنون برای انجام نگاشت، روند کلی ای که طی می شود به این شرح است. ابتدا ابعاد نگاشت در هر دو صفحه مشخص می شود. مختصات نقاط کالیبراسیون از قبل معلوم بود و مختصات مراکز عنبیه کاربر در حین نگاه به تصاویر هم به دست آورده شد. با استفاده از نسبت گیری بین اضلاع نظیر در مستطیل حاصله از اتصال مراکز کالیبراسیون و

مستطیل حاصله از اتصال مراکز عنیه، نسبت نگاشت به دست می‌آید. نسبت یا گام نگاشت به این معنی است که جابجایی مرکز عنیه به اندازه یک پیکسل، معادل جابجایی نگاه شخص به اندازه چند پیکسل در صفحه نمایش است. با به دست آوردن گام نگاشت، بخش کالیبراسیون صفحه نمایش با چشم کاربر به پایان می‌رسد و اکنون برنامه می‌تواند با دریافت تصاویر مختلف چشم کاربر، نقطه نگاه وی روی صفحه نمایش را در تصاویر مختلف تخمین بزند. روش برنامه برای تخمین محل نگاه کاربر به این صورت است که با دانستن گام نگاشت، فاصله مؤلفه‌های مرکز عنیه را با مؤلفه‌های نقطه بالا-چپ در کالیبراسیون به دست آورده و در گام نگاشت ضرب می‌کند تا متوجه شود که هر مؤلفه از مرکز دایره عنیه روی صفحه نمایش با مولفه نظیرش در نقطه بالا-چپ چه فاصله‌ای دارد. در انتها با جمع کردن مقدار این فاصله‌ها با مؤلفه‌های نقطه بالا-چپ، نقطه‌ای که کاربر به آن نگاه می‌کرده است را حدس می‌زند.



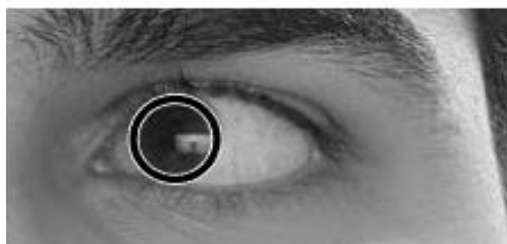
1820 , 100



1820 , 980



شکل ۳-۸: یکی از تصاویر کالیبراسیون آزمایشی. مرکز بعلاوه قرمز رنگ محلی است که کاربر به آن نگاه می‌کند و تصویربرداری می‌شود.

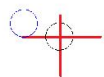


شکل ۳-۹: تصویر کاربر در حال نگاه به نقاط .

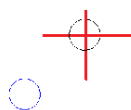


شکل ۳-۱۰: تصویر کاربر در حال نگاه به نقطه پنجم .

همان طور که در شکل های ۳-۹ و ۳-۱۰ قابل مشاهده است ، عنیبه با دقت خوبی پیدا شده است. خروجی کد و تخمین آن از محل نگاه کاربر در صفحه بعد آورده شده است.



شکل ۳-۱۱: تصویر نهایی اجرای کد برای تصاویر سری اول. دایره واقع در مرکز بعلاوه محلی است که کاربر نگاه کرده است و دایره آبی رنگ حدس کد از نگاه کاربر است.



شکل ۳-۱۲: تصویر نهایی اجرای کد برای تصاویر سری دوم.

در هر دو شکل ۱۱-۳ و ۱۲-۳ دایره مشکی رنگ مرکز بعلاوه قرمز رنگ، محل نگاه کاربر و دایره آبی رنگ در کنار آن حدس کد می باشد. علتی که در شکل ۱۱-۳ نزدیک تر حدس زده شده است این است که در یکی از تصاویر سری دوم اندکی چانه لرزیده است و همین مسئله باعث شده است تا حدس دقیق نشود.

چندین بار برای چند نقطه متفاوت این برنامه اجرا شد که میزان خطای پیکسلی که همان فاصله نقطه واقعی و نقطه حدس است برای ۶ سری از داده ها محاسبه شد. سری اول تصاویر ۳ درصد خطا، سری دوم تصاویر ۶ درصد خطا، سری سوم تصاویر ۳ درصد خطا، سری چهارم تصاویر ۹ درصد خطا، سری پنجم تصاویر ۸ درصد خطا و سری ششم تصاویر ۱۰ درصد خطا در حدس وجود داشت. همچنین در حالتی که چانه لرزید، ۱۶ درصد خطا محاسبه شد. لازم به ذکر است خطا، از تقسیم فاصله بین نقطه واقعی و نقطه حدس بر قطر صفحه نمایش به دست آورده شد. صفحه نمایش مورد استفاده 1920×1080 پیکسل بوده است.

۱-۲-۳ نتیجه گیری فاز دوم و برنامه های آینده

به طور کلی اجرای دقیق الگوریتم فاز دوم نیازمند ثابت بودن کامل چانه در طول اخذ تصاویر و همچنین تصویربرداری از پایین و تصویربرداری به صورت خود کار می باشد. داده های این فاز با استفاده از دوربین خویش انداز و به صورت دستی گرفته شد و ابزاری برای ثابت نگه داشتن چانه موجود نبود و از کاربر خواسته شد تا خود حرکتی نکند. در صورت فراهم بودن دوربین تصویربرداری خود کار در فواصل زمانی منظم یا با کلیک کاربر بر روی موشواره، می توان با اتصال به متلب این فاز را به صورت بلادرنگ و برای تعداد نقاط حدس بیشتری نیز انجام داد. همچنین می توان در صورت فراهم شدن این امکان، کاربردهای پیشرفته تری از قبیل یک بازی ساده حفظ توپ با حرکت چشم یا معرفی جایگزین موشواره برای این فاز طراحی و معرفی کرد.

- [1] A. Marion, Introduction to Image Processing, Springer, 2013.
- [2] R. C. Gonzalez, R. E. Woods and S. L. Eddins, Digital Image Processing Using MATLAB, Pearson Education, 2009.
- [3] R. Yadav, "Top 7 Image Processing Libraries In Python," Analytics India Magazine, 17 11 2019. [Online]. Available: <https://analyticsindiamag.com/top-8-image-processing-libraries-in-python/>.
- [4] K. K. Sunil and P. V. Arun, "Comparative analysis of common edge detection techniques in context of object extraction," *IEEE Transactions on Geoscience and Remote Sensing*, vol. 50, no. 11b, pp. 68-78, 2012.
- [5] V. Torre and T. A. Poggio, "On Edge Detection," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, Vols. PAMI-8, no. 2, pp. 147-163, 1986.
- [6] M. Bennamoun, "Edge Detection : Problems and Solutions," in *IEEE International Conference on Systems, Man, and Cybernetics. Computational Cybernetics and Simulation*, Orlando, 1997.
- [7] H. Mehrabian, A. Poursaberi and B. N. Araabi, "Iris boundary detection for iris recognition using Laplacian mask and Hough transform," in *4th Machine Vision and Image Processing conference*, 2007.
- [8] R. Jain, R. Kasturi and B. G. Schunck, "Edge Detection," in *Machine Vision*, Tempa, McGraw-Hill, 1995, pp. 140-185.

- [9] B. Wang and S. Fan, "An Improved CANNY Edge Detection Algorithm," in *Second International Workshop on Computer Science and Engineering*, Qingdao, 2009.
- [10] P. Zhou, W. Ye, Y. Xia and Q. Wang, "An Improved Canny Algorithm for Edge Detection," *Journal of Computational Information Systems*, vol. 5, no. 75, pp. 1516-1523, 2011.
- [11] D. Purves, G. J. Augustine, D. Fitzpatrick, L. C. Katz, A.-S. LaMantia, J. O. McNamara and M. Williams, *Neuroscience*, Sunderland, MA: Sinauer Associates, 2001.
- [12] R. P. Wildes, "Iris recognition: an emerging biometric technology," *Proceedings of the IEEE*, vol. 85, no. 9, pp. 1348-1363, 1997.
- [13] A. Garg and S. Chawla, "Comparative Survey of Various Iris Recognition," *International Journal of Computer, Communication and Information Technology*, vol. 1, no. 1, 2013.
- [14] C.-l. Tisse, L. Martin, L. Torres and M. Robert, "Person identification technique using human iris recognition," *Vision Interface*, vol. 294, no. 299, 2002.
- [15] S. S. Harakannanavar and V. I. Puranikmath, "Comparative Survey of Iris Recognition," in *International Conference on Electrical, Electronics, Communication, Computer and Optimization Techniques*, Mysuru, India, 2017.
- [16] L. G. Shapiro and G. C. Stockman, "Image Segmentation," in *Computer Vision*, Pearson, 2000, pp. 305-340.
- [17] R. O. Duda and H. E. Peter, "Use of the Hough transformation to detect lines and curves in pictures," *Communications of the ACM*, vol. 15, no. 1, pp. 11-15, 1972.

- [18] S. J. K. Pedersen, "Circular Hough Transform".
- [19] S. Wyder, F. Hennings, S. Pezold, J. Hrbacek and P. C. Cattin, "With Gaze Tracking Towards Noninvasive Eye Cancer Treatment," *IEEE Transactions on Biomedical Engineering*, vol. 63, no. 9, pp. 1914-1924, 2016.
- [20] P. S. Holzman, L. R. Proctor, D. L. Levy, N. J. Yassillo, H. Y. Meltzer and S. W. Hurt, "Eye-Tracking Dysfunctions in Schizophrenic Patients and Their Relatives," *JAMA Psychiatry*, vol. 31, no. 2, pp. 143-151, 1974.
- [21] K. Meena, M. Kumar and M. Jangra, "Controlling Mouse Motions Using Eye Tracking Using Computer Vision," in *Proceedings of the International Conference on Intelligent Computing and Control Systems*, 2020.
- [22] M. Q. Khan and S. Lee, "Gaze and Eye Tracking: Techniques and Applications in ADAS," *Sensors(Basel)*, vol. 19, no. 24, 2019.
- [23] J. Daugman, "How iris recognition works," in *The essential guide to image processing*, Academic Press, 2009, pp. 715-739.
- [24] M. A. El-Sayed, S. F. Bahgat and S. Abdel-Khalek, "Novel Approach of Edges Detection for Digital Images Based On Hybrid Types of Entropy," *Applied Mathematics & Information Sciences*, vol. 7, no. 5, pp. 1809-1817, 2013.